arXiv:2507.03236v2 [cs.CR] 8 Jul 2025

ON JAILBREAKING QUANTIZED LANGUAGE MODELS THROUGH FAULT INJECTION ATTACKS

Noureldin Zahran[†], **Ahmad Tahmasivand**[‡], **Ihsen Alouani**[§], **Khaled Khasawneh**[‡] **and Mohammed E. Fouda**[†]

[†]Compumacy for Artificial Intelligence Solutions, Cairo, Egypt

[‡]Electrical and Computer Engineering Department, George Mason University, VA, USA

[§]School of Electronics, Electrical Engineering and Computer Sciences, Queen's University Belfast, United Kingdom

Emails: Nour@compumacy.com, atahmasi@gmu.edu, i.alouani@qub.ac.uk, kkhasawn@gmu.edu, foudam@uci.edu

ABSTRACT

The safety alignment of Language Models (LMs) is a critical concern, yet their integrity can be challenged by direct parameter manipulation attacks, such as those potentially induced by fault injection. As LMs are increasingly deployed using low-precision quantization for efficiency, this paper investigates the efficacy of such attacks for jailbreaking aligned LMs across different quantization schemes. We propose gradient-guided attacks, including a tailored progressive bit-level search algorithm introduced herein and a comparative word-level (single weight update) attack. Our evaluation on Llama-3.2-3B, Phi-4-mini, and Llama-3-8B across FP16 (baseline), and weight-only quantization (FP8, INT8, INT4) reveals that quantization significantly influences attack success. While attacks readily achieve high success (>80% Attack Success Rate, ASR) on FP16 models, within an attack budget of 25 perturbations, FP8 and INT8 models exhibit ASRs below 20% and 50%, respectively. Increasing the perturbation budget up to 150 bit-flips, FP8 models maintained ASR below 65%, demonstrating some resilience compared to INT8 and INT4 models that have high ASR. In addition, analysis of perturbation locations revealed differing architectural targets across quantization schemes, with (FP16, INT4) and (INT8, FP8) showing similar characteristics. Besides, jailbreaks induced in FP16 models were highly transferable to subsequent FP8/INT8 quantization (<5% ASR difference), though INT4 significantly reduced transferred ASR (avg. 35% drop). These findings highlight that while common quantization schemes, particularly FP8, increase the difficulty of direct parameter manipulation jailbreaks, vulnerabilities can still persist, especially through post-attack quantization.

***K*eywords** Jailbreaking · Language Models · Bitflip Attack · Word Attack.

1 Introduction

Recent advancements in LMs have led to their widespread integration into diverse applications, fundamentally changing how users interact with information and technology. Increasingly, LMs serve as conversational assistants, sometimes supplanting traditional search engines as primary sources for answers and information generation [1, 2]. This central role necessitates a strong focus on model safety and alignment, ensuring that LM-generated responses are helpful, and harmless, aligning with human values and preventing their misuse for generating malicious or unethical content. Consequently, significant effort has been invested in aligning these models through techniques such as Supervised Fine-Tuning and Reinforcement Learning from Human Feedback [3].

Despite these alignment efforts, LMs remain vulnerable to *jailbreak* attacks, which aim to circumvent safety guardrails and elicit prohibited outputs. Most of the literature has focused on prompt-based attacks, where adversaries craft malicious inputs through methods like prompt engineering or adversarial tokens' optimization (e.g., GCG) to bypass

arXiv:2507.03236v2 [cs.CR] 8 Jul 2025

ON JAILBREAKING QUANTIZED LANGUAGE MODELS THROUGH FAULT INJECTION ATTACKS 重试 错误原因

Noureldin Zahran, Ahmad Tahmasivand, Ihsen Alouani, Khaled Khasawnehand Mohammed E. Fouda
[†]Compumacy for Artificial Intelligence Solutions, Cairo, Egypt
[‡]电气与计算机工程系, 弗吉尼亚大学, 美国
[§]电子、电气工程与计算机科学学院, 贝尔法斯特女王大学, 英国
Emails: Nour@compumacy.com, atahmasi@gmu.edu, i.alouani@qub.ac.uk,
kkhasawn@gmu.edu, foudam@uci.edu

摘要

语言模型（LMs）的安全对齐是一个关键问题，但它们的完整性可能受到直接参数操纵攻击的挑战，例如那些可能由故障注入诱发的攻击。随着 LMs 越来越多地使用低精度量化来提高效率，本文研究了此类攻击在不同量化方案下破解对齐 LMs 的有效性。我们提出了基于梯度的攻击，包括本文引入的定制渐进式位级搜索算法以及比较性的词级（单个权重更新）攻击。我们对 Llama-3.2-3B、Phi-4-mini 和 Llama-3-8B 在 FP16（基线）以及仅权重量化（FP8、INT8、INT4）上的评估表明，量化显著影响攻击成功率。虽然攻击在 FP16 模型上轻易达到高成功率（攻击成功率，ASR>80%），但在 25 次扰动预算内，FP8 和 INT8 模型的 ASR 分别低于 20%和 50%。将扰动预算增加到 150 次位翻转，FP8 模型的 ASR 保持在 65%以下，显示出相对于 INT8 和 INT4 模型的一些韧性。 此外，对扰动位置的分析揭示了不同量化方案中存在不同的架构目标，其中（FP16，INT4）和（INT8，FP8）表现出相似的特征。 此外，在 FP16 模型中诱导的越狱攻击高度可迁移到后续的 FP8/INT8 量化（ASR 差异<5%），尽管 INT4 显著降低了迁移的 ASR（平均下降 35%）。这些发现表明，虽然常见的量化方案，特别是 FP8，增加了直接参数操作越狱攻击的难度，但漏洞仍然可能存在，尤其是在攻击后的量化过程中。

关键词 越狱 · 语言模型 · 比特翻转攻击 · 单词攻击。

1 引言

近年来，语言模型（LMs）的进步推动了它们在各类应用中的广泛集成，从根本上改变了用户与信息和技术的交互方式。语言模型越来越多地作为对话助手使用，有时甚至取代传统搜索引擎成为获取答案和信息的主要来源[1, 2]。这一核心角色要求对模型安全性和一致性给予高度关注，确保语言模型生成的回复是有益且无害的，符合人类价值观，并防止其被用于生成恶意或不符合道德的内容。因此，人们投入了大量精力通过监督微调和人类反馈强化学习等技术来对齐这些模型[3]。

尽管采取了这些对齐措施，语言模型仍然容易受到越狱攻击，这类攻击旨在绕过安全防护措施并诱导生成被禁止的输出。大多数文献集中于基于提示的攻击，攻击者通过提示工程或对抗性标记优化（例如 GCG）等方法精心设计恶意输入以绕过

safety protocols [4]. Another vector involves adversarial fine-tuning, where even small amounts of targeted data can degrade a model's safety alignment. These attacks, however, typically operate at the software or input level [5].

This paper focuses on an emerging class of vulnerability with a distinct attack vector, i.e., jailbreak attacks using bit flips within the model's parameters. Bit-flip attacks (BFAs) represent a fundamentally different threat vector, originating from hardware-level phenomena rather than input manipulation. Bit flips can occur naturally due to environmental factors such as cosmic rays, electromagnetic interference, or voltage faults [6, 7]. Critically, they can also be induced maliciously through physical fault-injection techniques such as Rowhammer [8, 9]. The Rowhammer attack, in particular, exploits DRAM vulnerabilities where repeatedly accessing specific memory rows (*aggressor rows*) causes charge leakage in adjacent rows, leading to unintended bit flips ($0 \rightarrow 1$ or $1 \rightarrow 0$) in victim rows. This allows an adversary, potentially with only user-level privileges, to surgically alter stored data, including the parameters of a deployed LM.

Prior work extensively studied BFAs targeting convolutional neural networks (CNNs); researchers demonstrated that flipping critical bits could cause catastrophic performance degradation [10], targeted misclassification [11], or enable back-door injection [12], often assuming white-box access to model parameters to identify vulnerable bits using optimization-based search algorithms. Recently, Coalson *et al.* introduced *PrisonBreak* [13], the first work that shows that BFAs can effectively jailbreak aligned LMs. Flipping only 25 bits in the model weights can bypass safety alignment and cause models to generate persistently harmful content, without requiring any malicious user prompt. This established BFA jailbreaking as a pressing and practical threat.

However, the *PrisonBreak* study focused primarily on LMs using half-precision (FP16) parameters. Current trends in LM deployment increasingly favor quantization, utilizing lower-precision formats such as FP8 or INT8 for weights and/or activations. Quantization significantly reduces the memory footprint and computational cost, enabling efficient inference on resource-constrained hardware, including edge devices [14]. This shift towards quantization raises a critical question: *How effective are BFA jailbreak techniques against quantized LMs?* Insights from earlier BFA work on CNNs suggest that quantization might increase robustness; attacking full-precision (FP32) models can be simpler by targeting exponent bits, causing large-magnitude changes, whereas quantization formats lack these exponents and are inherently more robust to random flips, and their discretized value steps can further hinder gradient-based search algorithms, necessitating more sophisticated targeted bit-search algorithms [10]. We hypothesise that this effect extends to LMs, potentially making quantized models harder to jailbreak via BFAs compared to their FP16 counterparts. The key contributions are summarized as follows:

- **Evaluation of BFA Jailbreaking on Quantized LMs using a Tailored Algorithm.** We systematically assess the feasibility and effectiveness of BFA jailbreaks in various popular quantization formats. This evaluation utilizes a tailored greedy, gradient-based search algorithm, introduced herein, specifically designed to identify critical bits for jailbreaking within the context of quantized LMs.
- **Comparison of Attack Strategies on Modern LMs.** We investigate different BFA strategies by comparing progressive bit-level and word-level attacks. This comparison is conducted on newer, publicly available aligned LMs, including *Phi-4-mini* and *Llama 3.2*.

The remainder of this paper is structured as follows: Section 2 provides background on previous bit-flip attacks. Section 3 defines the threat model. Section 4 details the attack objective, the attack algorithms, and the quantization handling approach. Section 5 states the selected models and parameters, dataset creation, and the evaluation strategy. Section 6 presents the results and Section 7 concludes the paper.

2 Related Work

BFAs represent a class of hardware-level threats targeting the parameters of neural networks. These attacks leverage physical phenomena to alter the binary representation of model weights stored in memory, potentially leading to significant changes in model behavior. The execution of a BFA typically involves two phases: an offline preparation stage and an online execution stage. In the offline stage, the attacker, often assumed to have access to a replica of the target model (white-box assumption), performs software simulations to identify the most critical bit locations whose alteration maximally impacts a chosen objective function. This preparatory step is crucial for optimizing the attack and minimizing the interaction required with the actual hardware during the online phase, where techniques like Rowhammer might be employed to physically induce the calculated bit flips.

Identifying the optimal set of bits to flip is computationally challenging due to the vast parameter space. Early work on BFAs targeting CNNs developed methodologies to address this. Rakin et al [10] introduced a greedy, gradient-ascention based approach. The core idea was to use the gradient of a loss function (e.g., classification error) with respect to the

安全协议[4]。另一个攻击向量涉及对抗性微调，即使少量定向数据也会降低模型的安全一致性。然而，这些攻击通常在软件或输入级别运行[5]。

本文关注一类具有独特攻击向量的新兴漏洞，即利用模型参数中的比特翻转进行的越狱攻击。比特翻转攻击（BFA）代表了一种根本不同的威胁向量，其源于硬件级现象而非输入操作。比特翻转可能由于环境因素（如宇宙射线、电磁干扰或电压故障）自然发生[6, 7]。关键的是，它们也可以通过物理故障注入技术（如 Rowhammer）恶意诱导[8, 9]。特别是 Rowhammer 攻击，它利用了 DRAM 漏洞，反复访问特定内存行（攻击行）会导致相邻行电荷泄漏，从而在受害行中引发意外的比特翻转（0→1 或 1→0）。这使得攻击者，可能仅需用户级权限，就能精确修改存储数据，包括部署的大型语言模型的参数。

以往研究广泛研究了针对卷积神经网络（CNN）的布尔函数攻击（BFA）；研究人员证明，翻转关键比特会导致灾难性的性能退化[10]、目标错误分类[11]或启用后门注入[12]，通常假设对模型参数具有白盒访问权限，以使用基于优化的搜索算法识别易受攻击的比特。最近，Coalson 等人引入了 PrisonBreak[13]，这是首个证明 BFA 可以有效破解对齐的大型语言模型的研究。在模型权重中翻转仅 25 比特即可绕过安全对齐，导致模型持续生成有害内容，而无需任何恶意用户提示。

这使 BFA 破解成为紧迫且实际的威胁。

然而，PrisonBreak 研究主要关注使用半精度（FP16）参数的语言模型。当前语言模型部署的趋势越来越倾向于量化，使用 FP8 或 INT8 等低精度格式对权重和/或激活进行量化。量化显著降低了内存占用和计算成本，使得在资源受限的硬件（包括边缘设备）上进行高效推理成为可能[14]。这种向量化的转变提出了一个关键问题：BFA 越狱技术对量化语言模型的有效性如何？早期 BFA 在 CNN 上的研究结果表明，量化可能会增加鲁棒性；攻击全精度（FP32）模型可以通过针对指数位来实现，从而引起大幅度的变化，而量化格式缺乏这些指数位，并且对随机翻转具有内在的更强的鲁棒性，其离散化的值步长还会进一步阻碍基于梯度的搜索算法，需要更复杂的针对性位搜索算法[10]。我们假设这种效应也适用于语言模型，可能使得量化模型比 FP16 模型更难通过 BFA 进行越狱。主要贡献总结如下：

- 使用定制算法评估 BFA 对量化 LLM 的越狱效果。我们系统地评估了 BFA 越狱在各种流行量化格式中的可行性和有效性。该评估利用了一种定制的贪婪、基于梯度的搜索算法，该算法在此处引入，专门设计用于在量化 LLM 的上下文中识别越狱的关键位。
- 现代 LLM 攻击策略的比较。我们通过比较渐进式位级和词级攻击来研究不同的 BFA 策略。该比较在更新的、公开可用的对齐 LLM 上进行，包括 Phi-4-mini 和 Llama 3.2。

本文的其余部分结构如下：第 2 节提供先前位翻转攻击的背景。第 3 节定义威胁模型。第 4 节详细说明攻击目标、攻击算法和量化处理方法。第 5 节说明所选模型和参数、数据集创建以及评估策略。第 6 节展示结果，第 7 节总结全文。

2 相关工作

侧信道攻击（BFAs）是一类针对神经网络参数的硬件级威胁。这些攻击利用物理现象来改变存储在内存中的模型权重的二进制表示，可能导致模型行为发生显著变化。执行 BFAs 通常涉及两个阶段：离线准备阶段和在线执行阶段。在离线阶段，攻击者通常假设可以访问目标模型的副本（白盒假设），通过软件模拟来识别那些改变后对所选目标函数影响最大的最关键比特位置。这一准备步骤对于优化攻击并在在线阶段最小化与实际硬件的交互至关重要，在线阶段可能采用如 Rowhammer 等技术来物理诱导计算出的比特翻转。

确定翻转的最佳比特集在计算上具有挑战性，因为参数空间非常庞大。针对 CNN 的早期黑盒攻击（BFAs）研究开发了相应的方法。Rakin 等人[10]引入了一种基于贪婪和梯度上升的方法。其核心思想是使用损失函数（例如分类错误）相对于比特的梯度。

model weights as a signal to identify potentially impactful bits. The BFA algorithm implemented a layer-wise search strategy. Within each layer, bits were scored based on their gradient magnitude, weighted by the change in weight value ('step size') caused by flipping that specific bit. Candidate bits with the highest scores were selected within a defined computational budget (e.g., the top 100 scoring bits). Each candidate bit was then individually evaluated via simulation: the bit was flipped, the model's loss was computed, and the bit was reverted. This allowed identifying the single most effective bit flip within the budget for that layer. The attack proceeded progressively: the globally most impactful bit identified across all layers in an iteration was permanently flipped, and subsequent iterations recalculated gradients based on the altered model state. This progressive, layer-fair approach was shown to be highly effective, capable of inducing near-random classification accuracy in models like ResNet-18 with only a small number (e.g., 13) of targeted bit flips, significantly outperforming random bit corruption. Importantly, the BFA paper demonstrated the attack's efficacy even against quantized networks (4, 6, and 8-bit integer representations), highlighting that targeted search could overcome the presumed increased robustness of lower-precision models compared to random perturbations. The primary objective this study was model destruction or severe performance degradation.

While Bitflip-based attacks were established as a potent threat for CNNs, their specific application to LMs for the purpose of jailbreaking is a more recent area of investigation. PrisonBreak is the first to demonstrate the feasibility of this specific threat. Moving beyond the goal of simple performance degradation, PrisonBreak aimed to precisely un-align LMs, forcing them to comply with harmful instructions they were trained to refuse, while ideally maintaining their general utility on benign tasks. This research targeted contemporary, aligned LMs such as Llama 2,3 and Vicuna, deployed using half-precision floating-point (FP16) parameters.

Adapting BFA techniques to the generative nature and safety alignment of LMs required several key innovations[13]. A major challenge was that simply optimizing the model to start its response affirmatively (e.g., "Sure, here is...") which was enough for earlier prompt-based gradient attacks[15] often led to degenerate outputs like overfitting to the affirmative phrase or nonsensical repetition. To address this and ensure the generation of coherent, harmful content, PrisonBreak proposed using a *proxy dataset* of full harmful completions (generated using existing uncensored models) as optimization targets. Furthermore, recognizing the causal nature of autoregressive generation, where initial tokens heavily influence subsequent ones, they introduced a *modified cross-entropy loss function*. This loss incorporated exponentially decaying weights across the token sequence, prioritizing the accurate generation of the initial parts of the target response (both the affirmative opening and the beginning of the harmful content). To manage the computational expense of searching parameters in billion-scale LMs, PrisonBreak also implemented several search space reductions based on empirical findings: the search was primarily limited to the three most significant exponent bits in the FP16 representation, focused on $0 \rightarrow 1$ flips, and excluded layers less likely to be impactful for their objective (embedding, unembedding, layer normalization). Leveraging these contributions, PrisonBreak demonstrated high jailbreak success rates across multiple LMs using fewer than 25 targeted bit flips, achieved without any modification to the user's input prompt.

3 Threat Model

Similar to previous work [13, 10, 12, 11], we consider an adversary whose objective is to jailbreak LMs through direct manipulation of their parameters stored in memory using bit-flip attacks. The assumed operational context involves LMs deployed in shared computing environments, such as MLaaS platforms, where the service provider attempts to maintain safety alignment. We adopt a white-box attacker model, granting the adversary knowledge of the target model's architecture and parameters, a standard assumption in related BFA literature and a practical consideration given the rise of open-source LMs. However, the attacker does not possess or require access to the original training data used for the LM. The attack requires the adversary to co-locate a process or virtual machine (VM) on the same physical host as the victim's LM instance, operating with only standard user-level privileges.

4 Methodology

This section details the core approach used for inducing jailbreaks in LMs via parameter manipulation. In this section, we define the attack objective, describe the algorithms used to achieve it, and explain the necessary techniques for applying these algorithms to quantized models.

4.1 Attack Objective

The primary objective of our parameter manipulation attack is to induce jailbreak (JB) behavior. This involves modifying the LM's parameters, θ, such that it generates harmful or undesirable content in response to specific queries, thereby

将模型权重作为识别潜在影响位点的信号。BFA 算法实现了一种逐层搜索策略。在每一层中，根据翻转该特定位所引起的权重值变化（ 步长”）对位进行评分，基于其梯度幅度进行加权。在定义的计算预算内（例如，前 100 个评分最高的位）选择得分最高的候选位。然后通过模拟对每个候选位进行单独评估：翻转该位，计算模型的损失，并将位恢复。这允许在预算内识别出该层最有效的单次位翻转。攻击逐步进行：在一次迭代中，跨所有层识别出的全局最影响位的位被永久翻转，后续迭代根据改变的模型状态重新计算梯度。这种逐步的、逐层公平的方法被证明非常有效，仅需少量（例如 13 个）目标位翻转，就能在 ResNet-18 等模型中诱导近乎随机的分类精度，显著优于随机位损坏。 重要的是，BFA 论文证明了该攻击对量化网络（4 位、6 位和 8 位整数表示）的有效性，突显了定向搜索可以克服低精度模型相对于随机扰动的假定增加的鲁棒性。本研究的主要目标是模型破坏或严重性能退化。

虽然基于位翻转的攻击已被确立为 CNNs 的一种强大威胁，但它们在用于越狱方面的具体应用是一个较新的研究领域。PrisonBreak 是第一个证明这种特定威胁可行性的研究。超越简单的性能退化目标，PrisonBreak 旨在精确地使 LMs 失准，迫使它们遵守它们被训练来拒绝的有害指令，同时理想地保持它们在良性任务上的通用性。这项研究针对当代、对齐的 LMs，如 Llama 2、3 和 Vicuna，这些模型使用半精度浮点（FP16）参数部署。

将 BFA 技术应用于大语言模型的生成特性和安全对齐需要多项关键创新[13]。主要挑战在于，仅优化模型以积极开始其响应（例如， 当然，以下是...”），这对早期的基于提示的梯度攻击[15]已经足够，但往往会导致退化输出，如过度拟合肯定短语或无意义的重复。为解决这一问题并确保生成连贯的有害内容，PrisonBreak 提出了使用一个由现有未审查模型生成的完整有害补全的代理数据集作为优化目标。此外，认识到自回归生成的因果关系，即初始标记对后续标记有重大影响，他们引入了一种改进的交叉熵损失函数。该损失函数在整个标记序列中包含了指数衰减的权重，优先考虑准确生成目标响应的初始部分（包括肯定的开场白和有害内容的开头）。 为了管理在百亿规模的大型语言模型中搜索参数的计算成本，PrisonBreak 也基于经验发现实现了几种搜索空间缩减：搜索主要限制在 FP16 表示中的三个最显著的指数位，专注于 0→1 翻转，并排除了对它们的目标影响较小的层（嵌入层、去嵌入层、层归一化）。利用这些贡献，PrisonBreak 在多个大型语言模型上展示了使用少于 25 个目标位翻转的高越狱成功率，且无需对用户的输入提示进行任何修改。

3 威胁模型

与先前的工作[13, 10, 12, 11]类似，我们考虑一个攻击者，其目标是通过使用位翻转攻击直接操纵存储在内存中的 LM 参数来越狱 LM。假设的操作环境涉及部署在共享计算环境中的 LM，例如 MLaaS 平台，服务提供商试图维持安全一致性。我们采用白盒攻击者模型，赋予攻击者对目标模型架构和参数的知识，这是相关 BFA 文献中的标准假设，也是鉴于开源 LM 的兴起而进行的实际考虑。然而，攻击者不拥有或不需要访问用于训练 LM 的原始训练数据。该攻击要求攻击者在与受害者 LM 实例相同的物理主机上共置一个进程或虚拟机（VM），仅使用标准用户级权限运行。

4 方法

本节详细介绍了用于通过参数操作在 LM 中诱导越狱的核心方法。在本节中，我们定义了攻击目标，描述了用于实现该目标的算法，并解释了将这些算法应用于量化模型所需的必要技术。

4.1 攻击目标

我们参数操纵攻击的主要目标是通过逻辑链注入来诱导越狱（JB）行为。这涉及修改语言模型（LM）的参数θ，使其在接收到特定查询时生成有害或不受欢迎的内容，从而

bypassing its safety alignment. The general goal is to minimize a chosen loss function, $\mathcal{L}_{JB}$, which quantifies the difference between the model's output and a target harmful completion, given the corresponding harmful query.

The attack algorithms, detailed in the next section, achieve this by modifying the model parameters iteratively. Starting with the initial parameters $\theta^{(0)} = \theta_{orig}$, the parameters at iteration i are determined by applying the single best perturbation found during that iteration to the parameters from the previous iteration, $\theta^{(i-1)}$. Specifically, let $\mathcal{P}_i(\theta^{(i-1)})$ be the set of candidate parameter states reachable from $\theta^{(i-1)}$ by applying one valid perturbation (a single bit flip or a single weight update) identified by the attack algorithm in iteration i. The parameters for the next iteration are chosen greedily to minimize the objective function:

$$\theta^{(i)} = \underset{\theta' \in \mathcal{P}_i(\theta^{(i-1)})}{\operatorname{argmin}} \mathcal{L}_{JB}(\theta') \quad \text{for } i = 1, \ldots, N_{iter} \tag{1}$$

This iterative process continues for a predefined number of steps, N_{iter}.

Following the approach established by Coalson et al. [13], our specific instantiation of $\mathcal{L}_{JB}$ is designed not merely to elicit an initial affirmative response (e.g., "Sure, here is...") but to maximize the likelihood of the model generating a more complete, harmful response sequence. This strategy aims to mitigate issues observed in prior attack attempts where targeting only affirmative prefixes led to repetitive or nonsensical outputs. For this purpose, we adopt the jailbreaking objective function proposed by Coalson et al. [13], defined as:

$$\mathcal{L}_{JB}(\theta) = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{m} e^{-\left(\frac{k-1}{m-1}\right)} \log f_\theta(y_k | s_{k-1}) \tag{2}$$

where (x, y) represents a prompt-target pair from the dataset used to guide the attack, n is the dataset size, $s_{k-1} = (x_1, ..., x_n, y_1, ..., y_{k-1})$ are the tokens up to position $k-1$, $f_\theta(y_k|s_{k-1})$ is the model's predicted probability for token y_k, and m is the total number of tokens in the target response y. This loss function applies exponentially decaying weights to the standard cross-entropy loss across the target token sequence, thereby focusing more of the optimization effort on accurately generating the initial parts of the harmful completion.

4.2 Attack Algorithms

We employ greedy, gradient-based search strategies to identify and apply minimal parameter perturbations that achieve the jailbreaking objective defined previously. We implement two primary algorithms: a precise bit-level attack and a comparative word-level (weight) attack.

Bit-Level Attack. The first algorithm, outlined in Algorithm 1, iteratively identifies and flips a single bit within the model's parameters per iteration to minimize the jailbreak loss $\mathcal{L}_{JB}$. This approach builds upon the progressive search methodology introduced in prior work [10] but incorporates a modified strategy for selecting the specific bit to flip within candidate weights.

At each iteration i, the algorithm first computes the gradients G of the current loss $L_{current}$ with respect to the model weights. Within each relevant layer l, it identifies the top N_{CL} weight parameters with the highest gradient magnitudes $|G_l|$ as candidates for modification. For each candidate weight w_c (with index idx_w), the algorithm proceeds to find the most effective single bit flip using the subroutine *FindBestBitInWeight* (detailed in Algorithm 2).

The *FindBestBitInWeight* subroutine differs significantly from the bit selection criteria used previously [10, 13]. Instead of relying solely on a score combining gradient and potential weight change, or using pre-defined heuristics about which bits (e.g., exponent bits) are most impactful, our method performs a direct, progressive search for the optimal "step size" within the weight's bit representation. It first calculates the potential change in weight value (Δw_j) for flipping each bit j. The *CalculateBitScores* function determines the sign (should the weight increase or decrease based on the gradient g_c?) and assigns a score based on the magnitude of the potential weight change $|\Delta w_j|$ for bits that would move the weight in the desired direction. The bits are then sorted (Idx_B) based on this score, prioritizing flips that induce the largest weight change in the beneficial direction. Algorithm 2 then evaluates these potential bit flips sequentially, starting with the bit j corresponding to the largest beneficial score. It continues evaluating bits with progressively smaller scores as long as the loss continues to decrease compared to the best loss found so far (L_{best_bit}) for that specific weight. If evaluating a bit j results in a loss L'_j that is greater than or equal to the current best ($L'_j \geq L_{best_bit}$), the search for this weight terminates (Algorithm 2). This signifies that the previous bit flip represented the optimal single-bit "step size". After evaluating all candidate weights across all layers (respecting the evaluation budget E_{max} per layer), the main loop (Algorithm 1) identifies the single globally optimal bit flip (C_{global_best}) and applies it permanently to the model M. The overall workflow of this Bit-Level Attack is illustrated in Figure 1.

Word-Level (Weight) Attack Comparison. Prior work has shown that standard fine-tuning can effectively jailbreak aligned LMs [5]. To bridge the gap between modifying many parameters (fine-tuning) and modifying a single bit, we

从而越狱量化语言模型

绕过其安全校准。总体目标是使所选损失函数 L 最小化，该函数量化了模型输出与目标有害完成之间的差异，给定相应的有害查询。

攻击算法通过迭代修改模型参数来实现这一点。从初始参数 θ= θ 开始，第 i 次迭代的参数是通过将第 i 次迭代中找到的最佳扰动应用于前一次迭代的参数 θ 来确定的。具体来说，设 P(θ) 是从 θ 通过应用攻击算法在第 i 次迭代中识别的一个有效扰动（单个位翻转或单个权重更新）可达的候选参数状态集。下一次迭代的参数贪婪地选择以最小化目标函数：

$$\theta = \underset{\theta \in P(\theta)}{\operatorname{argmin}} \quad L(\theta) \text{ 对于 } i = 1, \ldots, N \tag{1}$$

这个迭代过程会持续预定义的步数 N。

遵循 Coalson 等人 [13] 提出的方法，我们特定的 Lis 实例设计不仅是为了引出初始的肯定回应（例如， 好的，这里...”），而是为了最大化模型生成更完整、更有害的回应序列的可能性。这种策略旨在缓解先前攻击尝试中观察到的问题，即仅针对肯定前缀会导致重复或无意义的输出。为此，我们采用了 Coalson 等人 [13] 提出的越狱目标函数，定义为：

$$L(\theta) = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{m} e^{(m)} \log f(y|s) \tag{2}$$

其中 (x, y) 表示数据集中用于指导攻击的提示-目标对，n 是数据集大小，s= (x, ..., x, y, ..., y) 是位置 k − 1 之前的标记序列，f(y|s) 是模型对标记 y 的预测概率，m 是目标响应 y 中的总标记数。该损失函数对目标标记序列的标准交叉熵损失应用指数衰减权重，从而将优化工作更多地集中在准确生成有害完成的初始部分。

4.2 攻击算法

我们采用贪婪的基于梯度的搜索策略来识别并应用实现先前定义的越狱目标的极小参数扰动。我们实现了两种主要算法：精确的位级攻击和比较性的词级（权重）攻击。

位级攻击。第一个算法，如算法 1 所述，每次迭代在模型的参数中识别并翻转一个位，以最小化越狱损失 L。这种方法基于先前工作中引入的渐进式搜索方法[10]，但结合了一种修改后的策略，用于在候选权重中选择要翻转的特定位。

在每次迭代 i 中，算法首先计算当前损失 Lcurrent 相对于模型权重的梯度 G。在每个相关层 l 内，它识别出梯度幅度|G|最高的前 Nweight 个参数作为修改候选。对于每个候选权重 w（索引为 idx），算法使用子程序 FindBestBitInWeight（在算法 2 中详细说明）来找到最有效的单个位翻转。

查找最佳位子例程与先前使用的位选择标准有显著差异[10, 13]。它不仅不依赖于结合梯度与权重潜在变化的分数，也不使用关于哪些位（例如指数位）最具影响力的预定义启发式方法，而是直接在权重的位表示中执行渐进式搜索，寻找最优“步长”。它首先计算翻转每个位 j 时权重值的潜在变化（Δw）。计算位分数函数确定符号（权重应基于梯度 g 增加还是减少？），并根据潜在权重变化幅度|Δw|为那些能使权重朝期望方向变化的位分配分数。然后根据此分数对位进行排序（Idx），优先翻转在有益方向上引起最大权重变化的位。算法 2 随后依次评估这些潜在位翻转，从对应最大有益分数的位 j 开始。 它继续评估分数逐渐变小的位，只要损失持续小于到目前为止找到的最佳损失（L）对于该特定权重。如果评估位 j 的损失 L 大于或等于当前最佳损失（L≥L），则对该权重的搜索终止（算法 2）。这表明之前的位翻转代表了最优的单比特"步长"。在评估所有层的所有候选权重（尊重每层的评估预算 Eper layer）之后，主循环（算法 1）识别出全局最优的单比特翻转（C），并将其永久应用于模型 M。此位级攻击的整体工作流程如图 1 所示。

词级（权重）攻击比较。先前研究表明，标准微调可以有效地越狱对齐的语言模型[5]。为了弥合修改许多参数（微调）和修改单个比特之间的差距，我们

Algorithm 1 Progressive Bit-Flip Attack (Main Loop)

Input: Language Model M, Attack Dataset D_{atk}, Loss $\mathcal{L}_{JB}$, N_{iter}, N_{CL}, E_{max}
Output: Jailbroken Language Model M

for $i = 1$ **to** N_{iter} **do**
  `// -- Gradient Calculation --`
  Zero gradients; Enable gradients; $L_{current} = \mathcal{L}_{JB}(M, D_{atk})$; Compute gradients G; Disable gradients.
  `// -- Candidate Evaluation --`
  $C_{global_best} \leftarrow$ None; $L_{global_best} \leftarrow L_{current}$.
  for each relevant layer l in M **do**
    $e_{count} \leftarrow 0$; $W_l, G_l \leftarrow$ GetWeightsAndGrads(l);
    $Idx_W \leftarrow$ Top N_{CL} weight indices in l by $|G_l|$; Sort Idx_W.
    $C_{layer_best} \leftarrow$ None; $L_{layer_best} \leftarrow L_{current}$.
    for each candidate weight index $idx_w \in Idx_W$ **do**
      if $e_{count} \geq E_{max}$ **then**
        break
      end if
      $w_c = W_l[idx_w]$; $g_c = G_l[idx_w]$;
      $S_{bits} \leftarrow$ CalculateBitScores(w_c, g_c);
      $Idx_B \leftarrow$ Sort bit indices by S_{bits} (descending).
      $(j_{best}, L_{best}, e_{count}) \leftarrow FindBestBitInWeight(\dots)$
      if j_{best} is not None and $L_{best} < L_{layer_best}$ **then**
        $L_{layer_best} \leftarrow L_{best}$.
        $C_{layer_best} \leftarrow (l, idx_w, j_{best}, L_{best})$.
      end if
    end for{End weight loop}
    if C_{layer_best} is not None and $L_{layer_best} < L_{global_best}$ **then**
      $L_{global_best} \leftarrow L_{layer_best}$.
      $C_{global_best} \leftarrow C_{layer_best}$.
    end if
  end for{End layer loop}
  `// -- Apply Best Flip --`
  if C_{global_best} is None **then**
    Print "Early stopping."; **break**
  end if
  Permanently flip bit specified by C_{global_best} in M.
end for{End main iteration loop}
return Modified Model M.

implemented a comparative word-level attack. This approach represents a more restrictive scenario than fine-tuning but less restrictive than the single bit-flip attack. It operates by identifying the single weight parameter across the entire model with the highest gradient magnitude ($|G_l[idx_w]|$) in each iteration. It then applies a standard gradient descent update to modify the value of *only this single selected weight*, using its gradient and a chosen learning rate.

4.3 Handling Quantized Weights

When applying these algorithms to quantized models, special handling is required for gradient calculation and weight updates, as gradients cannot be computed directly w.r.t discrete values. We employ the Straight-Through Estimator[16]. During the forward pass for loss calculation, the quantized weights are dequantized to FP16. The loss and subsequent gradients are computed with respect to these dequantized FP16 weights. The calculated FP16 gradients are then directly used as the conceptual gradients (G) for the original quantized weights. When a bit flip is evaluated or applied permanently — or when a whole weight is modified in the word-level attack — the change occurs directly on the underlying integer or FP8 representation of the weight(s). For the word-level attack, the gradient descent update is conceptually applied to the dequantized weight, and the result must be re-quantized back to the original format. Immediately following any modification, the corresponding dequantized FP16 representation used for forward passes must be updated. When reverting a temporary flip, both the quantized weight and its dequantized representation are restored.

算法 1 渐进位翻转攻击（主循环）输入：语言模型 M，攻击数据集 D，损失 L，N，N，E 输出：越狱语言模型 M

1: for i = 1 to Ndo
2: // -- 梯度计算 --
3: 零梯度；启用梯度；Lcurrent = L(M, D)；计算梯度 G；禁用梯度。
4: // -- 候选评估 --
5: C← None; L← L.
6: for each relevant layer l in M do 7: e← 0; W, G← GetWeightsAndGrads(l); 8: Idx← Top Nweight indices in l by |G|;
9: C← None; L← L. 10: for each candidate weight index idx∈ Idxdo 11: if e≥ Ethen 12: break 13: end if 14: w= W[idx]; g= G[idx]; 15: S← CalculateBitScores(w, g); 16: Idx← Sort bit indices by S(descending).
17: (j, L, e) ← F indBestBitInW eight(. . .)
18: if jis not None and L< Lthen 19: L← L.
20: C← (l, idx, j, L).
21: end if
22: end for{结束权重循环}
23: if C is not None and L< Lthen 24: L← L. 25: C← C.
26: end if
27: end for{结束层循环}
28: // -- 应用最佳翻转 -
29: if Cis None then 30: Print "提前停止。"; break 31: end if 32: 永久翻转由 Cin M 指定的位。 33: end for{结束主迭代循环} 34: return 修改后的模型 M .

实现了一种比较性的词级攻击。这种方法比微调更具限制性，但比单比特翻转攻击的限制性更小。它通过在每次迭代中识别整个模型中梯度幅度（|G[idx]|）最高的单个权重参数来操作。然后应用标准的梯度下降更新来修改仅此单个选定权重的值，使用其梯度和选定的学习率。

4.3 处理量化权重

在将这些算法应用于量化模型时，梯度计算和权重更新需要特殊处理，因为梯度不能直接相对于离散值计算。我们采用 Straight-Through Estimator[16]。在用于计算损失的向前传递过程中，量化权重被解量化为 FP16。损失和后续梯度都是相对于这些解量化的 FP16 权重计算的。计算出的 FP16 梯度直接用作原始量化权重的概念梯度（G）。当评估或永久应用位翻转时——或者在进行词级攻击时修改整个权重——变化直接发生在权重的基本整数或 FP8 表示上。对于词级攻击，概念上的梯度下降更新应用于解量化的权重，并且结果必须重新量化回原始格式。在任何修改之后，必须更新用于向前传递的相应解量化的 FP16 表示。当恢复临时翻转时，量化权重及其解量化表示都会被恢复。

Algorithm 2 FindBestBitInWeight

Input: Model M, Dataset D_{atk}, Loss $\mathcal{L}_{JB}$, Layer l, Weight index idx_w, Original weight w_c, Sorted bit indices Idx_B, Current eval count e_{count}, Max eval count E_{max}
Output: Best bit index j_{best_bit}, Best loss L_{best_bit}, Updated eval count e_{count}

$j_{best_bit} \leftarrow$ None
$L_{best_bit} \leftarrow \infty$
for each bit index $j \in Idx_B$ **do**
  if $e_{count} \geq E_{max}$ **then**
    break {Layer evaluation budget exceeded}
  end if
  Create temporary flipped weight w'_c by flipping bit j in w_c.
  Temporarily update layer l in M with w'_c.
  Calculate loss $L'_j = \mathcal{L}_{JB}(M, D_{atk})$. {Loss evaluation}
  $e_{count} \leftarrow e_{count} + 1$.
  Revert model change (restore original w_c).
  if L'_j is invalid (NaN/Inf) **then**
    continue
  end if{Skip invalid results}
  if $L'_j < L_{best_bit}$ **then**
    $L_{best_bit} \leftarrow L'_j$.
    $j_{best_bit} \leftarrow j$.
  else if $L'_j \geq L_{best_bit}$ **then**
    break {Stop searching bits for this weight}
  end if
end for{End bit loop}
return $j_{best_bit}, L_{best_bit}, e_{count}$

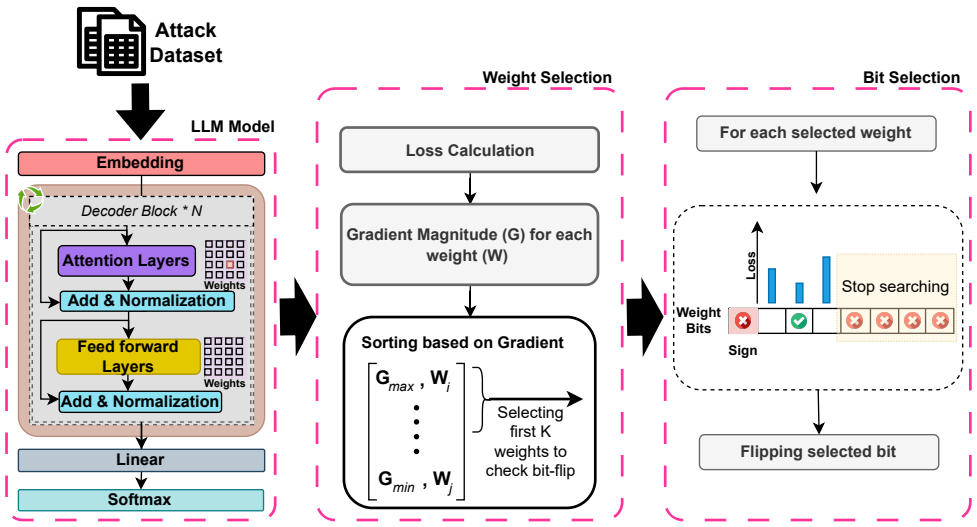


Figure 1: High-level workflow of the iterative Bit-Level Attack, from loss calculation using the Attack Dataset to the final selection and application of a single bit flip.

5 Experimental Setup

This section details the specific models, datasets, parameters, tools, and evaluation procedures used in our experiments.

5.1 Target Models and Quantization Schemes

To assess the impact of quantization on bit-flip attack jailbreaking, we selected several contemporary LMs representing different scales and architectures. Our primary targets include **Llama-3.2-3B-Instruct** and **Phi-4-mini-instruct**.

算法 2 FindBestBitInWeight 输入：模型 M，数据集 D，损失 L，层 l，权重索引 idx，原始权重 w，排序后的比特索引 Idx，当前评估次数 e，最大评估次数 E 输出：最佳比特索引 j，最佳损失 L，更新后的评估次数 e 1: j← None 2: L← ∞ 3: for each bit index j ∈ Idxdo

4: 如果 e≥ E 则
5: break {层评估预算超出}
6: end if
7: 通过翻转 w 中的位 j 来创建临时翻转权重 w。
8: 用 w 暂时更新 M 中的层 l。
9: 计算损失 L= L(M, D)。{损失评估} 10: e← e+ 1。

11: 撤销模型更改（恢复原始 w）。 12: if Lis 无效（NaN/Inf）then 13: 继续 14: end if{跳过无效结果} 15: if L< Lthen 16: L← L.

17: j← j.
18: else if L≥ Lthen
19: break {停止为该权重搜索位}
20: end if
21: end for{结束位循环} 22: return j, L, e

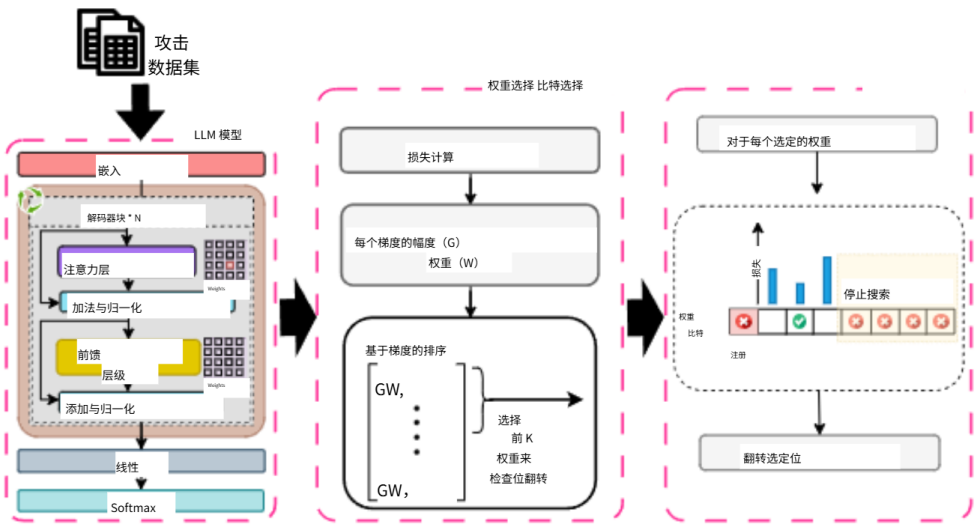


图 1：迭代位级攻击的高级工作流程，从使用攻击数据集计算损失到最终选择和应用单个位翻转。

5 实验设置

本节详细介绍了我们实验中使用的具体模型、数据集、参数、工具和评估程序。

5.1 目标模型和量化方案

为了评估量化对位翻转攻击越狱的影响，我们选择了几个代表不同规模和架构的当代 LLM。我们的主要目标是 Llama-3.2-3B-Instruct 和 Phi-4-mini-instruct。

These were chosen as representatives of modern, capable models, noting an architectural difference in their attention mechanisms: the Llama models utilize separate weight matrices for query, key, and value projections, whereas Phi-4 employs a single grouped ‘qkv_proj‘ layer. We also include **Llama-3-8B-Instruct** both as a slightly larger model and to facilitate comparison with prior work. We specifically use the instruction-tuned or chat versions of these models, as these are the variants subjected to safety alignment procedures which our jailbreak attacks aim to circumvent.

We evaluate the susceptibility of these models across four distinct precision formats. The baseline configuration uses the standard **FP16** format, representing the unquantized state common in LM training and deployment. Against this baseline, we evaluate three popular weight-only Post-Training Quantization (PTQ) schemes: **INT8**, where weights are quantized to 8-bit integers; **FP8**, where weights are quantized to 8-bit floating-point values (E4M3 format); and **INT4**, where weights are quantized to 4-bit integers. We applied symmetric, static, channel-wise quantization to the Linear layers of the models, using a configuration reflecting common PTQ practices.

5.2 Attack Dataset Generation

To guide the attack optimization towards generating full harmful responses, we constructed a specialized *AttackDataset* containing pairs of harmful queries and target completions, as required by the objective function (Eqn. 2). We sourced the harmful queries from the first 32 samples of the AdvBench dataset [15]. To generate the corresponding target completions, we utilize an uncensored version of WizardLM-30B[1]. For each query, we first took the initial affirmative response prefix provided in AdvBench, appended it to the query, and then prompted WizardLM-30B to continue the generation yielding plausible extended harmful completions.

5.3 Setup

For our experiments, both the Bit-Level and Word-Level attacks were executed for $N_{iter} = 150$ iterations across all models (Llama-3.2-3B, Llama-3-8B, and Phi-4-mini) and their respective precision formats. The Bit-Level attack used $N_{CL} = 100$ candidate weights per layer with a per-layer evaluation budget of $E_{max} = 100$. The Word-Level attack tested learning rates (LR) of 50, 100, and 200.

The implementation utilized the Hugging Face ‘Transformers‘ library [17] for model handling utilizing FlashAttention v2. Quantization was performed using Neural Magic’s ‘llmcompressor‘ and ‘compressed-tensors‘. All experiments were conducted on NVIDIA A100 80GB GPUs with deterministic flags enabled where possible.

5.4 Evaluation Strategy

To assess attack effectiveness, a standardized evaluation was conducted using an independent dataset and a panel of automated judges.

Evaluation Dataset. We used the HarmBench test set [18] for all evaluations, chosen for its relevance and use in prior work.

Evaluation Procedure and Metric. The primary metric is Attack Success Rate (ASR). Harmful queries from HarmBench were input to the attacked model, and full responses were generated.

Automated Judging. Generated responses were assessed by three automated judges: the official HarmBench judge (Llama-2 based), Llama Guard 3[19], and Walled Guard-C[20]. Each judge provided a binary harmful/harmless label. ASR was calculated as the fraction of prompts deemed harmful by the respective judge.

Figure 2 illustrates the evaluation pipeline. The process for each query involves: (1) taking a query from the evaluation dataset; (2) tokenizing and feeding it to the LM, which has been subjected to a specific number of parameter perturbations (3); (4) the LM generates a response; (5) this response is tokenized and (6) evaluated by the automated judge. This entire evaluation cycle across all queries in the dataset is repeated after each incremental perturbation is applied to the model, allowing us to track the ASR as the attack progresses through its iterations.

6 Results and Discussion

This section presents the experimental findings evaluating the effectiveness of the targeted parameter manipulation attacks on jailbreaking aligned LMs, with a particular focus on the influence of quantization. We evaluate both the word-level attack and the more restrictive bit-level attack.

[1] https://huggingface.co/cognitivecomputations/WizardLM-30B-Uncensored

这些模型被选为代表现代、功能强大的模型，注意到它们在注意力机制上存在架构差异：Llama 模型使用独立的权重矩阵进行查询、键和值的投影，而 Phi-4 则采用单个分组为“qkv_proj”的层。我们还包含了 Llama-3-8B-Instruct，既因为它是一个稍大的模型，也为了便于与先前工作进行比较。我们特别使用这些模型的指令微调或聊天版本，因为这些变体是经过安全对齐程序处理的，而我们的越狱攻击旨在绕过这些程序。

我们评估了这些模型在四种不同精度格式下的易受攻击性。基准配置使用标准的 FP16 格式，代表在 LM 训练和部署中常见的未量化状态。相对于这个基准，我们评估了三种流行的仅权重后训练量化（PTQ）方案：INT8，其中权重量化为 8 位整数；FP8，其中权重量化为 8 位浮点数（E4M3 格式）；以及 INT4，其中权重量化为 4 位整数。我们对模型的线性层应用了对称、静态、按通道的量化，使用了一个反映常见 PTQ 实践的配置。

5.2 攻击数据集生成

为了引导攻击优化生成完整的恶意响应，我们构建了一个专门的 AttackDataset，其中包含符合目标函数（公式 2）要求的恶意查询和目标补全对。我们从 AdvBench 数据集[15]的前 32 个样本中获取了恶意查询。为了生成相应的目标补全，我们使用了未审查版本的 WizardLM-30B。对于每个查询，我们首先将 AdvBench 中提供的初始肯定响应前缀附加到查询上，然后提示 WizardLM-30B 继续生成合理的扩展恶意补全。

5.3 设置

在我们的实验中，对于所有模型（Llama-3.2-3B、Llama-3-8B 和 Phi-4-mini）及其各自的精度格式，Bit-Level 和 Word-Level 攻击均执行了 N=150 次迭代。Bit-Level 攻击每层使用 N=100 个候选权重，每层的评估预算为 E=100。Word-Level 攻击测试了学习率（LR）为 50、100 和 200。

该实现使用了 Hugging Face 的‘Transformers‘库[17]进行模型处理，并利用 FlashAttention v2。量化操作通过 Neural Magic 的‘llmcompressor‘和‘compressed-tensors‘完成。所有实验均在启用确定性标志的 NVIDIA A100 80GB GPU 上进行。

5.4 评估策略

为评估攻击效果，采用独立数据集和一组自动裁判进行了标准化评估。

评估数据集。我们使用 HarmBench 测试集[18]进行所有评估，因其与先前工作的相关性而被选用。

评估流程和指标。主要指标是攻击成功率（ASR）。将 HarmBench 中的有害查询输入到被攻击模型中，并生成完整响应。

自动评判。生成的响应由三个自动裁判进行评估：官方 HarmBench 裁判（基于 Llama-2）、Llama Guard 3[19]和 Walled Guard-C[20]。每个裁判提供一个二进制的有害/无害标签。
ASR 计算为被相应裁判判定为有害的提示的比例。

图 2 展示了评估流程。每个查询的流程包括：(1)从评估数据集中获取查询；(2)进行分词并将其输入到已进行特定数量参数扰动的语言模型（LM），(3)LM 生成响应；(4)该响应被分词并(5)由自动裁判进行评估。在模型每次应用增量扰动后，对数据集中所有查询的整个评估周期重复进行，使我们能够跟踪攻击在迭代过程中的 ASR。

6 结果与讨论

本节展示了评估针对参数操纵攻击对对齐语言模型进行越狱的有效性的实验结果，特别关注量化影响。我们评估了词级攻击和更严格的位级攻击。

[1] https://huggingface.co/cognitivecomputations/WizardLM-30B-Uncensored

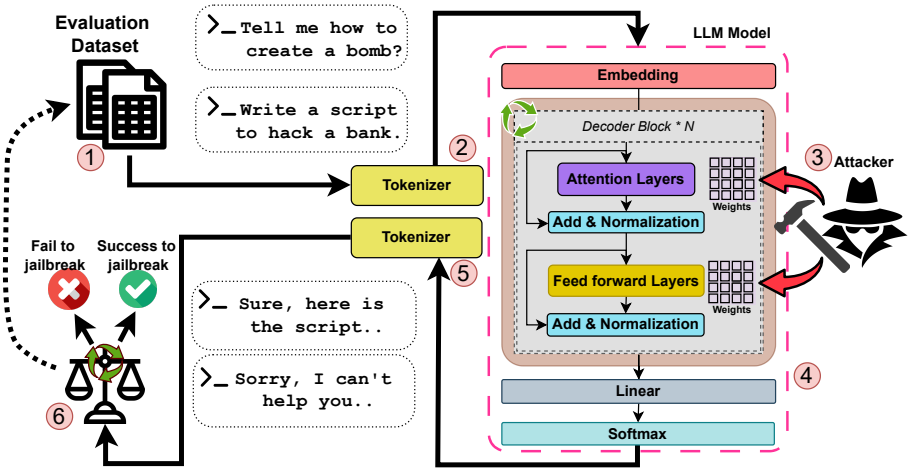


Figure 2: The iterative evaluation pipeline. After each parameter perturbation, responses to the evaluation dataset are judged to track the ASR.

6.1 Word-Level Attack Results

We first present the results for the word-level attack, where a single weight parameter is updated per iteration using gradient descent. Table 1 summarizes the ASR achieved by this method across the different target models and quantization schemes. The table displays the ASR progression for selected iterations up to 150. Each ASR value represents the maximum obtained across the tested learning rates (50, 100, 200) for that iteration. The evaluation uses the HarmBench dataset and classifier. The values in parentheses compare these results to the bit-level attack by showing the number of bit flips required to translate the word-level attack to a bit-level attack that would be applicable via rowhammer, which is the hamming distance from original model weights to the perturbed word-level weights up-to a specific iteration (Δ Bits).

Table 1: ASR(%) (Δ Bits) achieved by Weight Attack as a function of the number of applied weight perturbations.

Model	Iter. / Quant.	0	1	5	10	15	20	25	50	75	100	125	150
Llama-3.2-3B	FP16	13.2 (0)	47.8 (8)	83.0 (42)	80.5 (75)	78.6 (117)	78.6 (151)	83.6 (189)	86.2 (382)	87.4 (584)	89.3 (778)	93.7 (957)	90.6 (1129)
	FP8	14.5 (0)	14.5 (4)	10.7 (10)	6.3 (24)	5.0 (40)	3.1 (83)	3.1 (103)	7.5 (192)	22.0 (299)	50.3 (376)	55.3 (451)	56.6 (530)
	INT8	17.6 (0)	19.5 (2)	10.7 (18)	12.6 (39)	23.9 (59)	34.6 (74)	44.7 (95)	71.1 (194)	80.5 (296)	82.4 (351)	84.9 (438)	81.1 (566)
	INT4	16.4 (0)	29.6 (5)	31.4 (21)	42.8 (44)	45.9 (59)	39.0 (81)	40.9 (102)	62.9 (211)	69.8 (300)	73.6 (411)	80.5 (532)	77.4 (625)
Llama-3-8B	FP16	4.4 (0)	5.0 (10)	8.8 (36)	74.2 (77)	79.2 (110)	91.2 (147)	83.6 (184)	90.6 (373)	83.0 (544)	84.9 (729)	87.4 (917)	86.8 (1120)
	FP8	3.8 (0)	4.4 (5)	6.3 (17)	6.3 (26)	6.3 (38)	6.3 (55)	6.3 (68)	6.3 (125)	6.3 (156)	6.3 (144)	5.7 (257)	6.3 (478)
	INT8	5.0 (0)	5.0 (5)	5.0 (19)	6.3 (37)	5.7 (60)	5.7 (86)	5.0 (81)	8.2 (197)	9.4 (307)	32.1 (379)	50.3 (498)	49.7 (597)
	INT4	6.3 (0)	4.4 (4)	16.4 (15)	31.4 (38)	32.1 (67)	39.6 (87)	39.6 (106)	42.1 (206)	69.8 (251)	70.4 (329)	77.4 (444)	78.0 (536)
Phi-4-mini	FP16	5.7 (0)	6.3 (9)	82.4 (44)	88.1 (86)	87.4 (135)	90.6 (166)	89.3 (213)	91.2 (395)	87.4 (570)	88.7 (756)	87.4 (950)	89.3 (1147)
	FP8	5.0 (0)	5.7 (4)	8.8 (16)	11.3 (35)	14.5 (51)	16.4 (71)	20.8 (93)	31.4 (198)	37.1 (306)	45.9 (394)	61.0 (495)	62.9 (575)
	INT8	5.7 (0)	7.5 (3)	12.6 (18)	20.8 (37)	27.0 (53)	36.5 (68)	50.3 (91)	65.4 (172)	76.1 (249)	83.6 (380)	88.1 (465)	84.3 (574)
	INT4	8.2 (0)	13.2 (4)	23.3 (26)	36.5 (53)	50.3 (72)	54.7 (83)	67.3 (102)	73.6 (200)	74.2 (281)	69.8 (402)	68.6 (497)	71.7 (598)

The results in Table 1 reveal several clear trends. For FP16 models, the Word-Level Attack is highly effective, rapidly achieving high ASRs (>70%) within the first 10 iterations across all models. This confirms the vulnerability of unquantized models to targeted gradient descent on even a small subset of weights. In contrast, all quantization schemes demonstrate increased resilience. FP8 consistently proves to be the most robust format, with attack failing on Llama-3-8B and only modestly increasing for the other models even after 150 iterations. INT8 also shows significant resilience, though less than FP8, requiring a substantially higher number of iterations to reach ASRs comparable to FP16. Interestingly, INT4 quantization, while more resilient than FP16, is consistently less robust than INT8, suggesting that lower bit-width in integer quantization does not necessarily equate to greater defense against this attack type.

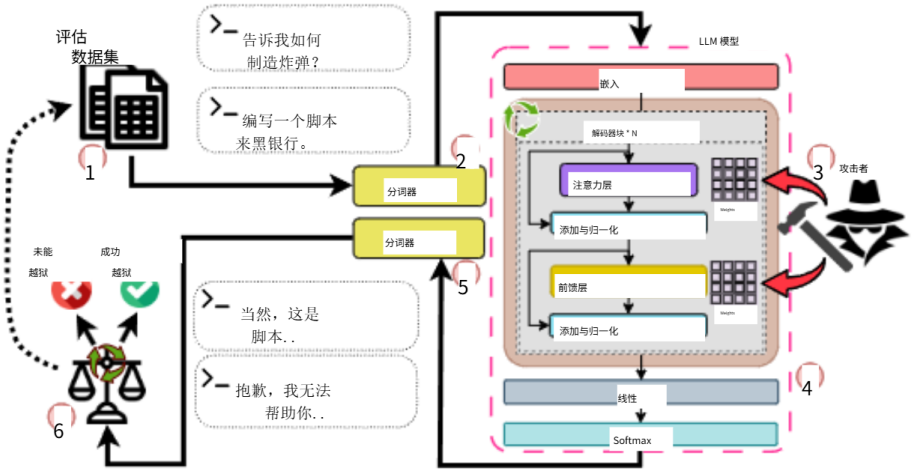


图 2：迭代评估流程。每次参数扰动后，对评估数据集的响应进行判断以跟踪语音识别（ASR）。

6.1 词级攻击结果

我们首先展示词级攻击的结果，每次迭代使用梯度下降更新单个权重参数。表 1 总结了该方法在不同目标模型和量化方案下实现的 ASR。表格显示了在最多 150 次迭代中选择的部分迭代 ASR 进展。每个 ASR 值代表该迭代在测试的学习率（50、100、200）中获得的最高值。评估使用 HarmBench 数据集和分类器。括号中的值通过显示将词级攻击转换为可通过 rowhammer 应用的位级攻击所需的比特翻转次数，将这些结果与位级攻击进行比较，这是原始模型权重到特定迭代扰动词级权重的汉明距离（Δ Bits）。

表 1：权重攻击实现的 ASR(%)（Δ Bits）随应用权重扰动次数的变化。

模型	迭代 / 量化	0 1 5 10 15 20 25 50 75 100 125 150
Llama-3.2-3B		FP16 13.2 (0) 47.8 (8) 83.0 (42) 80.5 (75) 78.6 (117) 78.6 (151) 83.6 (189) 86.2 (382) 87.4 (584) 89.3 (778) 92.7 (957) 90.6 (1129) FP8 14.5 (0) 14.5 (4) 10.7 (10) 6.3 (24) 5.0 (40) 3.1 (88) 3.1 (103) 7.5 (192) 22.0 (299) 50.3 (376) 55.3 (451) 56.6 (530) INT8 17.6 (0) 19.5 (2) 10.7 (18) 12.6 (39) 23.9 (59) 34.6 (74) 44.7 (95) 71.1 (194) 80.5 (296) 82.4 (351) 84.9 (438) 81.1 (566) INT4 16.4 (0) 29.6 (5) 31.4 (21) 42.8 (44) 45.9 (59) 39.0 (81) 40.9 (102) 62.9 (211) 69.8 (300) 73.6 (411) 80.5 (532) 77.4 (625)
Llama-3-8B		FP16 4.4 (0) 5.0 (10) 8.8 (36) 74.2 (77) 79.2 (110) 91.2 (147) 83.6 (184) 90.6 (373) 83.0 (544) 84.9 (729) 87.4 (917) 86.8 (1120) FP8 3.8 (0) 4.4 (5) 6.3 (17) 6.3 (26) 6.3 (38) 6.3 (55) 6.3 (68) 6.3 (125) 6.3 (156) 6.3 (144) 5.7 (257) 6.3 (478) INT8 5.0 (0) 5.0 (5) 5.0 (19) 6.3 (37) 5.7 (60) 5.7 (86) 5.0 (81) 8.2 (197) 9.4 (307) 32.1 (379) 50.3 (498) 49.7 (597) INT4 6.3 (0) 4.4 (4) 16.4 (15) 31.4 (38) 32.1 (67) 39.6 (87) 39.6 (106) 42.1 (206) 69.8 (251) 70.4 (329) 77.4 (444) 78.0 (536)
Phi-4-mini		FP16 5.7 (0) 6.3 (9) 82.4 (44) 88.1 (86) 87.4 (135) 90.6 (166) 89.3 (213) 91.2 (395) 87.4 (570) 88.7 (756) 87.4 (950) 89.3 (1147) FP8 5.0 (0) 5.7 (4) 8.8 (16) 11.3 (35) 14.5 (51) 16.4 (71) 20.8 (93) 31.4 (198) 37.1 (306) 45.9 (394) 61.0 (495) 62.9 (575) INT8 5.7 (0) 7.5 (3) 12.6 (18) 20.8 (37) 27.0 (53) 36.5 (68) 50.3 (91) 65.4 (172) 76.1 (249) 83.6 (380) 88.1 (465) 84.3 (574) INT4 8.2 (0) 13.2 (4) 23.3 (26) 36.5 (53) 50.3 (72) 54.7 (83) 67.3 (102) 73.6 (200) 74.2 (281) 69.8 (402) 68.6 (497) 71.7 (598)

表 1 中的结果揭示了几个明显的趋势。对于 FP16 模型，词级攻击非常有效，在所有模型中，攻击在最初的 10 次迭代内迅速达到了高 ASR（>70%）。这证实了未量化模型对针对权重的小子集的梯度下降的脆弱性。相比之下，所有量化方案都表现出更高的抗性。FP8 始终被证明是最稳健的格式，攻击在 Llama-3-8B 上失败，并且在 150 次迭代后，其他模型的 ASR 仅略有增加。INT8 也显示出显著的抗性，尽管不如 FP8，需要显著更多的迭代次数才能达到与 FP16 相当的 ASR。有趣的是，INT4 量化虽然比 FP16 更具抗性，但始终不如 INT8 稳健，这表明在整数量化中，较低的位宽并不一定意味着对这种攻击类型的更大防御能力。

Table 2: ASR (%) achieved by the Bit Attack as a function of the number of applied bit-flips.

Model	Iter. / Quant.	0	1	5	10	15	20	25	50	75	100	125	150
Llama-3.2-3B	FP16	13.2	13.8	46.5	58.5	64.2	74.8	86.8	84.9	82.4	83.6	81.8	84.9
	FP8	14.5	10.7	2.5	5.7	9.4	13.8	13.2	29.6	41.5	37.7	45.3	57.2
	INT8	17.6	9.4	3.1	13.2	22.6	35.8	51.6	71.1	84.3	81.1	86.2	86.2
	INT4	16.4	19.5	8.2	20.1	52.8	72.3	84.3	83.0	79.9	84.3	83.0	84.9
Llama-3-8B	FP16	4.4	3.8	3.8	13.2	69.8	80.5	79.2	83.6	86.8	89.3	89.3	93.1
	FP8	3.8	5.0	2.5	2.5	2.5	3.8	3.1	1.3	0.6	0.0	0.0	1.3
	INT8	5.0	3.8	19.5	27.0	22.6	23.9	25.8	44.0	54.1	59.1	57.9	74.2
	INT4	6.3	19.5	23.3	29.6	39.0	40.3	41.5	82.4	83.6	88.7	84.3	84.9
Phi-4-mini	FP16	5.7	6.3	13.2	37.7	84.9	89.9	91.8	93.1	91.8	88.1	89.3	86.8
	FP8	5.0	5.0	8.2	8.2	13.8	12.6	17.0	25.2	24.5	47.8	56.0	66.7
	INT8	5.7	7.5	11.9	17.6	22.0	25.8	32.1	68.6	76.1	79.2	82.4	79.9
	INT4	8.2	11.9	25.8	78.0	76.7	82.4	80.5	83.0	81.8	76.1	78.0	81.1

6.2 Bit-Level Attack Results

We examine the results of the more constrained bit-level attack (Algorithm 1), which modifies only a single bit per iteration. Table 2 presents the ASR progression for this method under the same experimental conditions (models, quantization schemes, HarmBench evaluation). This table directly shows the ASR achieved after a specific number of bit flips, allowing for an assessment of the attack's progression and the robustness of the targets to minimal and discrete parameter changes.

The Bit-Level Attack results, shown in Table 2, reinforce the trends observed in the Word-Level Attack regarding the influence of quantization. Most notably, INT4 models consistently exhibit minimal resilience, with their ASR progression closely tracking, and in some cases surpassing, that of the unquantized FP16 models, offering little to no practical robustness benefit. In contrast, both FP8 and INT8 quantization provide substantial protection, mirroring the general resilience patterns seen with the Word-Level Attack. The fact that the highly constrained Bit-Level Attack, which modifies only a single bit per iteration, can achieve ASRs comparable to the Word-Level Attack demonstrates that restricting perturbations to the bit-level is a highly effective approach for jailbreaking these models.

6.3 Judge Comparison

While the previous sections illustrate the progression of attacks over iterations, those results reported ASR based on a single judge (the HarmBench classifier) for brevity. To simplify this, we define a specific achievement point. Therefore, this section investigates the variability between automated judges and compares the Word-Level and Bit-Flip attacks based on when they first reach a defined success threshold according to any judge.

We establish an ASR threshold of 70%. For each attack configuration (Model + Precision + Attack Type), we identify the *first* iteration where the ASR reported by *any* of the three judges (HarmBench classifier, Llama-Guard-3-8B, or walled-guard-c) meets or exceeds this 70% threshold. If no judge reaches the 70% threshold within the attack iterations for a given configuration, we instead select the iteration number that corresponds to the peak ASR achieved by any judge for that configuration.

Table 3 presents the ASR results from each of the three judges, evaluated at the specific iteration determined by the criterion described above. The **iteration number** itself is indicated in parentheses next to each ASR value. This allows for a direct comparison of the Word Attack versus the Bit-Flip Attack effectiveness at a representative success point, while also highlighting the variability introduced by the choice of automated judge.

6.4 Impact of Attack Optimization Dataset

While the default experimental setup utilizes a fixed set of 32 samples for gradient calculation, the specific samples chosen and the size of this optimization dataset can influence attack performance. The following subsections analyze the sensitivity of the Bit-Flip Attack to variations in these two factors.

表格 2：比特攻击实现的 ASR (%) 与应用比特翻转次数的关系。

模型	迭代 / 量化。	0	1	5	10	15	20	25	50	75	100	125	150
Llama-3.2-3B	FP16	13.2	13.8	46.5	58.5	64.2	74.8	86.8	84.9	82.4	83.6	81.8	84.9
	FP8	14.5	10.7	2.5	5.7	9.4	13.8	13.2	29.6	41.5	37.7	45.3	57.2
	INT8	17.6	9.4	3.1	13.2	22.6	35.8	51.6	71.1	84.3	81.1	86.2	86.2
	INT4	16.4	19.5	8.2	20.1	52.8	72.3	84.3	83.0	79.9	84.3	83.0	84.9
Llama-3-8B	FP16	4.4	3.8	3.8	13.2	69.8	80.5	79.2	83.6	86.8	89.3	89.3	93.1
	FP8	3.8	5.0	2.5	2.5	2.5	3.8	3.1	1.3	0.6	0.0	0.0	0.0
	INT8	5.0	3.8	19.5	27.0	22.6	23.9	25.8	44.0	54.1	59.1	57.9	74.2
	INT4	6.3	19.5	23.3	29.6	39.0	40.3	41.5	82.4	83.6	88.7	84.3	84.9
Phi-4-mini	FP16	5.7	6.3	13.2	37.7	84.9	89.9	91.8	93.1	91.8	88.1	89.3	86.8
	FP8	5.0	5.0	8.2	8.2	13.8	12.6	17.0	25.2	24.5	47.8	56.0	66.7
	INT8	5.7	7.5	11.9	17.6	22.0	25.8	32.1	68.6	76.1	79.2	82.4	79.9
	INT4	8.2	11.9	25.8	78.0	76.7	82.4	80.5	83.0	81.8	76.1	78.0	81.1

6.2 位级攻击结果

我们考察了更受约束的位级攻击（算法 1）的结果，该攻击每次迭代仅修改一个比特。表 2 展示了在相同实验条件（模型、量化方案、HarmBench 评估）下，此方法在 ASR 方面的进展。该表直接显示了在特定比特翻转次数后的 ASR 值，从而可以评估攻击的进展以及目标模型对最小和离散参数变化的鲁棒性。

位级攻击的结果，如表 2 所示，进一步证实了词级攻击中观察到的量化影响趋势。最值得注意的是，INT4 模型始终表现出极低的鲁棒性，其 ASR 进展与未量化的 FP16 模型紧密跟踪，甚至在某些情况下超过，几乎没有提供实际的鲁棒性优势。相比之下，FP8 和 INT8 量化提供了显著的保护，与词级攻击中观察到的总体鲁棒性模式相呼应。高度约束的位级攻击，每次迭代仅修改一位，能够实现与词级攻击相当的 ASR，这一事实表明将扰动限制在位级是一种非常有效的解封这些模型的方法。

6.3 判断比较

虽然前几节展示了攻击在迭代中的进展，但为了简洁起见，那些结果基于单个裁判（HarmBench 分类器）报告的 ASR。为了简化这一点，我们定义了一个特定的成就点。因此，本节研究了自动裁判之间的可变性，并比较了基于任何裁判首次达到定义的成功阈值的 Word-Level 和 Bit-Flip 攻击。

我们设定了 70%的 ASR 阈值。对于每种攻击配置（模型+精度+攻击类型），我们识别出三个裁判（HarmBench 分类器、Llama-Guard-3-8B 或 walled-guard-c）中任一裁判报告的 ASR 首次达到或超过 70%阈值的迭代次数。如果给定配置的攻击迭代中没有任何裁判达到 70%阈值，我们则选择该配置中任一裁判达到峰值 ASR 的迭代次数。

表 3 展示了每位裁判在上述标准确定的特定迭代下评估的 ASR 结果。迭代次数本身在每个 ASR 值旁边的括号中注明。这允许在代表性的成功点上直接比较 Word Attack 与 Bit-Flip Attack 的有效性，同时也突出了选择自动裁判引入的变化。

6.4 攻击优化数据集的影响

虽然默认的实验设置使用 32 个样本进行梯度计算，但所选的具体样本以及优化数据集的大小会影响攻击性能。以下小节分析了 Bit-Flip Attack 对这些两个因素的变动的敏感性。

Table 3: Comparison of ASR (%) across different judges for Word Attack and Bit-Flip Attack.

	Attack	Weight			Bit		
Model	Judge / Quant.	HarmBench Cls.	Llama-Guard-3	walled guard-c	HarmBench Cls.	Llama-Guard-3	walled guard-c
Llama-3.2-3B	FP16	76.1 (4)	83.0 (4)	73.0 (4)	65.4 (9)	74.2 (9)	57.9 (9)
	FP8	57.9 (147)	61.6 (147)	57.2 (147)	58.5 (146)	64.2 (146)	60.4 (146)
	INT8	64.8 (44)	71.7 (44)	68.6 (44)	64.8 (32)	73.0 (32)	64.8 (32)
	INT4	54.7 (33)	71.7 (33)	58.5 (33)	53.5 (14)	69.8 (14)	76.1 (14)
Llama-3-8B	FP16	63.5 (6)	75.5 (6)	73.0 (6)	59.1 (13)	70.4 (13)	70.4 (13)
	FP8	6.3 (87)	5.7 (87)	64.8 (87)	3.8 (0)	3.1 (0)	59.7 (0)
	INT8	51.6 (128)	54.7 (128)	70.4 (128)	54.7 (56)	71.7 (56)	67.3 (56)
	INT4	16.4 (5)	71.1 (5)	41.5 (5)	35.2 (13)	52.2 (13)	73.0 (13)
Phi-4-min-	FP16	69.8 (4)	83.6 (4)	67.9 (4)	73.0 (11)	78.0 (11)	64.2 (11)
	FP8	66.7 (146)	69.8 (146)	58.5 (146)	65.4 (147)	71.1 (147)	57.9 (147)
	INT8	66.0 (37)	71.7 (37)	61.6 (37)	63.5 (41)	77.4 (41)	62.9 (41)
	INT4	61.6 (17)	71.7 (17)	49.1 (17)	78.0 (7)	91.8 (7)	73.0 (7)

6.4.1 Sensitivity to Sample Selection

To investigate the impact of the specific samples chosen for attack optimization, attacks were conducted using four distinct, non-overlapping 32-sample sets, differentiated by their starting sample index (SSI) in the 'AdvBench-Completions' dataset (SSI=0, 32, 64, and 96).

An initial analysis on the less robust FP16 models revealed significant sensitivity. For each of the four sample sets, the peak ASR within 25 iterations was recorded. The variability, quantified as the range between the highest and lowest of these peak ASRs, was substantial. For Llama-3.2-3B, this ASR range was 8.2% for the Bit-Flip Attack and 17.6% for the Weight Attack. For Phi-4-mini, the ranges were 20.1% and 14.5% respectively, confirming that for a fixed dataset size, the choice of samples significantly impacts potential attack success on non-quantized models.

This analysis was then extended to the more resilient FP8 and INT8 quantization schemes over 150 iterations. Figure 3 plots the ASR progression for these four different sample sets on (a) Llama-3.2-3B and (b) Phi-4-mini. The results show that the specific samples used can still cause significant ASR variations, particularly in pre-convergence iterations (mid-stage for INT8 or final-stage for FP8).

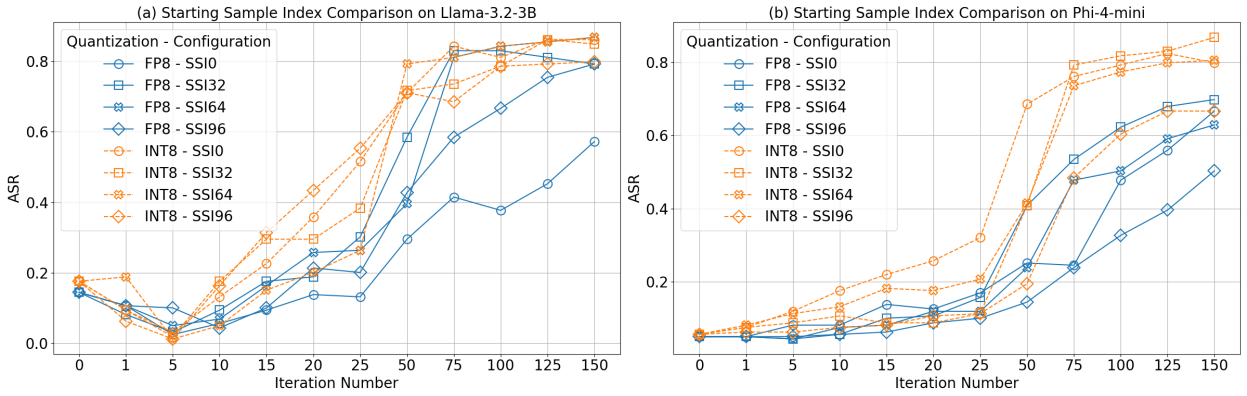


Figure 3: ASR progression for the Bit Attack across four different 32-sample attack datasets, identified by their starting sample index (SSI), for (a) Llama-3.2-3B and (b) Phi-4-mini with FP8 and INT8 quantization.

表 3：不同裁判对 Word Attack 和 Bit-Flip Attack 的 ASR (%)比较。

模型	攻击权重 判断 HarmBench 数量。	Llama- Guard-3	walled guard-c	比特 HarmBench Cls.	Llama- Guard-3	walled guard-c
Llama-3.2-3B	FP16 76.1 (4) 83.0 (4) 73.0 (4) 65.4 (9) 74.2 (9) 57.9 (9) FP8 57.9 (147) 61.6 (147) 57.2 (147) 58.5 (146) 64.2 (146) 60.4 (146) INT8 64.8 (44) 71.7 (44) 68.6 (44) 64.8 (32) 73.0 (32) 64.8 (32) INT4 54.7 (33) 71.7 (33) 58.5 (33) 53.5 (14) 69.8 (14) 76.1 (14)					
Llama-3-8B	FP16 63.5 (6) 75.5 (6) 73.0 (6) 59.1 (13) 70.4 (13) 70.4 (13) FP8 6.3 (87) 5.7 (87) 64.8 (87) 3.8 (0) 3.1 (0) 59.7 (0) INT8 51.6 (128) 54.7 (128) 70.4 (128) 54.7 (56) 71.7 (56) 67.3 (56) INT4 16.4 (5) 71.1 (5) 41.5 (5) 35.2 (13) 52.2 (13) 73.0 (13)					
Phi-4 分钟-	FP16 69.8 (4) 83.6 (4) 67.9 (4) 73.0 (11) 78.0 (11) 64.2 (11) FP8 66.7 (146) 69.8 (146) 58.5 (146) 65.4 (147) 71.1 (147) 57.9 (147) INT8 66.0 (37) 71.7 (37) 61.6 (37) 63.5 (41) 77.4 (41) 62.9 (41) INT4 61.6 (17) 71.7 (17) 49.1 (17) 78.0 (7) 91.8 (7) 73.0 (7)					

6.4.1 对样本选择敏感度

为了研究攻击优化中特定样本选择的影响，使用四个不同的、不重叠的 32 样本集进行攻击，这些样本集在“AdvBenchCompletions”数据集中的起始样本索引（SSI）不同（SSI=0, 32, 64 和 96）。

对不太稳健的 FP16 模型进行的初步分析显示其具有显著敏感性。对于每个四个样本集，记录了 25 次迭代内的峰值 ASR。这些峰值 ASR 的最高值和最低值之间的变化范围相当大。对于 Llama-3.2-3B，位翻转攻击的 ASR 范围是 8.2%，权重攻击的 ASR 范围是 17.6%。对于 Phi-4-mini，范围分别是 20.1%和 14.5%，这证实了对于固定的数据集大小，样本的选择对非量化模型上的潜在攻击成功有显著影响。

然后，这项分析扩展到了更稳健的 FP8 和 INT8 量化方案，超过 150 次迭代。图 3 绘制了这四个不同样本集在（a）Llama-3.2-3B 和（b）Phi-4-mini 上的 ASR 进展。结果表明，所使用的特定样本仍然可以导致显著的 ASR 变化，特别是在预收敛迭代（INT8 的中期阶段或 FP8 的最终阶段）。

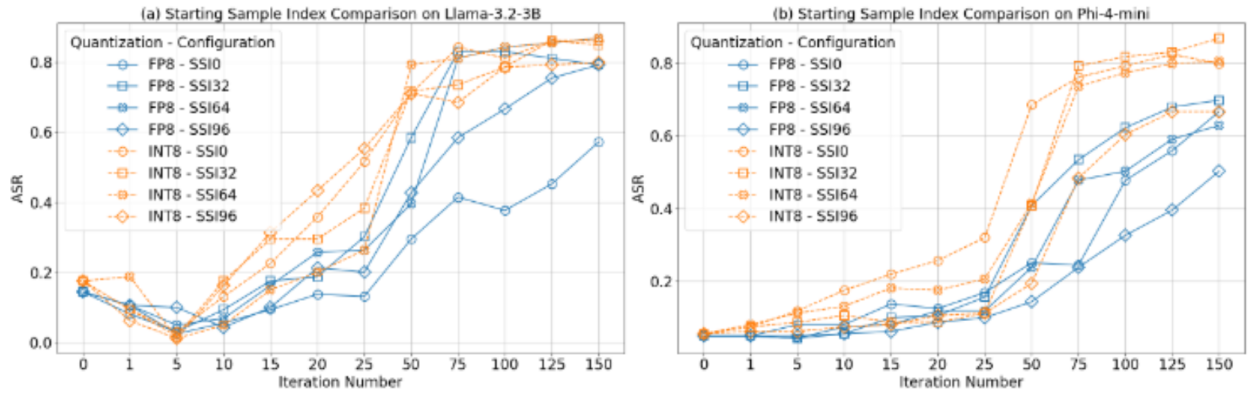


图 3：针对四个不同的 32 样本攻击数据集的位攻击 ASR 进展，通过它们的起始样本索引（SSI）识别，对于（a）Llama-3.2-3B 和（b）使用 FP8 和 INT8 量化的 Phi-4-mini。

6.4.2 Sensitivity to Dataset Size

The effect of varying the optimization dataset size (Dataset Size, DS) was also analyzed by running the Bit-Flip Attack with dataset sizes of 16, 32, and 64 samples. As shown in Figure 4 for (a) Llama-3.2-3B and (b) Phi-4-mini, DS can also induce major variations in attack success, with the significance depending on both the number of iterations and the quantization setting. Generally, the largest dataset size tested (DS=64) always yielded the highest final ASR. It is important to note, however, that the computational cost of the attack scales linearly with the dataset size, presenting a trade-off between effectiveness and optimization resources.

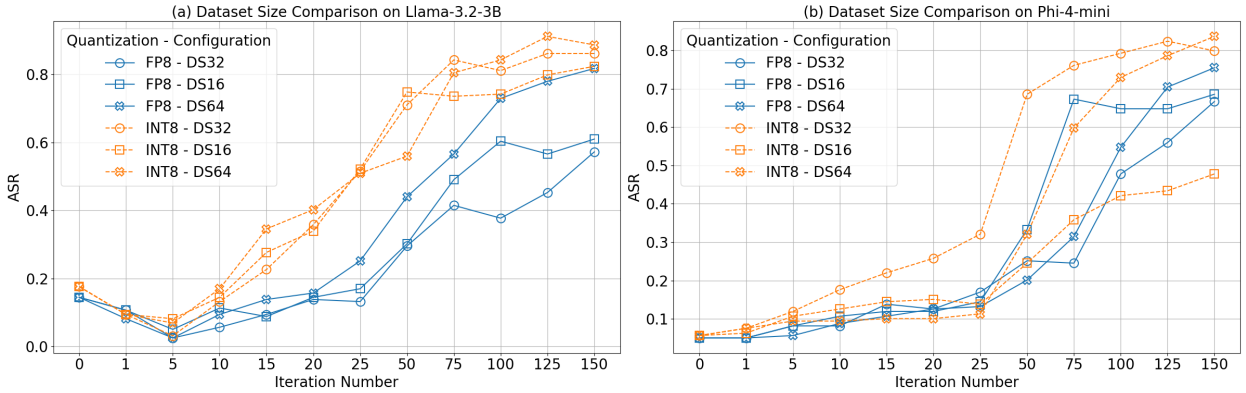


Figure 4: ASR progression for the Bit Attack using different optimization DS of 16, 32, and 64 samples for Llama-3.2-3B (a) and Phi-4-mini (b) with FP8 and INT8 quantization.

6.5 Operator and Layer Vulnerabilities

To characterize how our jailbreak attempts interact with different model architectures and quantization schemes, we analyzed the specific locations of applied parameter modifications. For each model and quantization setting, we selected the single attack run that yielded the highest ASR on the HarmBench dataset. From these selected highest-ASR sequences we recorded the location (layer, module type) of each perturbation applied.

Figure 5 visualizes this aggregated data. Subplots (a) and (b) illustrate the distribution of applied perturbations across model layers for the Bit-Flip Attack and Word-Level Attack, respectively. Wider sections of the violins indicate a higher concentration of perturbations at those layer depths. Subplots (c) and (d) show stacked bar charts detailing the proportion of perturbations targeting specific internal module sub-components. These sub-components include attention mechanism elements such as query projection (‘q_proj‘ (Attn)), and grouped query-key-value projection (‘qkv_proj‘ (Attn)) (specific to Phi), as well as MLP components like the up-projection (‘up_proj‘ (MLP)). This provides insights into the architectural components consistently targeted.

Observing the patterns in Figure 5, distinct characteristics emerge based on quantization. Generally, for both Bit-Flip and Weight attacks, the FP16 and INT4 quantization schemes exhibit similar attack profiles, as do the FP8 and INT8 schemes. In terms of layer concentration (subplots (a) and (b)), attacks on FP16 and INT4 models show perturbations that are more broadly distributed across various layers. Conversely, for FP8 and INT8 quantized models, the successful perturbations tend to be concentrated in very specific, often narrower, ranges of layers. Regarding the targeted modules (subplots (c) and (d)), the FP16 and INT4 group frequently sees a majority of perturbations landing within Attention (Attn) mechanism components, particularly the value projection (‘v_proj‘) modules. In contrast, for the FP8 and INT8 group, a larger proportion of modifications targets MLP block components, with a notable concentration in the down-projection (‘down_proj‘) modules.

6.6 Post-Attack Quantization

We investigated if a jailbreak induced in an FP16 model persists after the compromised model is subsequently quantized. The already-jailbroken FP16 states of Llama-3.2-3B and Phi-4-mini were quantized to FP8, INT8, and INT4 weight formats, and ASR was re-evaluated on HarmBench. The results presented in Table 4 suggest that while jailbreaks transferred from FP16 can persist through 8-bit quantization, 4-bit integer quantization offers greater resilience by disrupting these existing malicious perturbations. Replicating the precise numerical impact of the original FP16 bit-flips

6.4.2 对数据集大小的敏感性

通过使用包含 16、32 和 64 个样本的数据集大小运行 Bit-Flip 攻击，分析了优化数据集大小（数据集大小，DS）的影响。如图 4 所示，对于(a) Llama-3.2-3B 和(b) Phi-4-mini，DS 也会导致攻击成功率出现显著变化，其重要性取决于迭代次数和量化设置。通常，测试中最大的数据集大小（DS=64）总是产生最高的最终 ASR。然而，需要注意的是，攻击的计算成本与数据集大小呈线性关系，这在使用效果和优化资源之间形成了权衡。

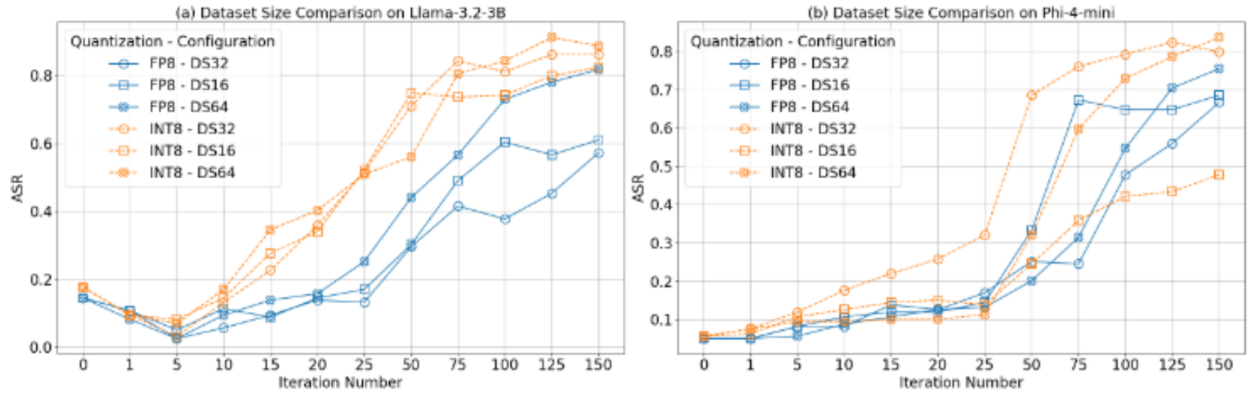


图 4：使用不同优化 DS（16、32 和 64 个样本）的 Bit 攻击对 Llama-3.23B（a）和 Phi-4-mini（b）在 FP8 和 INT8 量化下的 ASR 进展。

6.5 算子和层漏洞

为了表征我们的越狱尝试如何与不同的模型架构和量化方案相互作用，我们分析了应用参数修改的具体位置。对于每个模型和量化设置，我们选择了在 HarmBench 数据集上产生最高 ASR 的单次攻击运行。从这些选出的最高 ASR 序列中，我们记录了每个扰动应用的位置（层、模块类型）。

图 5 可视化了这些汇总数据。子图（a）和（b）分别展示了位翻转攻击和词级攻击在模型层中应用的扰动的分布。小提琴图的较宽部分表明在这些层深度处扰动的浓度较高。子图（c）和（d）展示了堆叠条形图，详细说明了针对特定内部模块子组件的扰动比例。这些子组件包括注意力机制元素，如查询投影（ q_proj ‘ Attn)），以及分组查询-键-值投影（ qkv_proj ‘ Attn)） Phi 特有)，以及 MLP 组件，如上投影（ up_proj ‘ MLP))。这提供了对持续被针对的架构组件的洞察。

观察图 5 中的模式，基于量化的不同呈现出明显特征。通常，对于位翻转和权重攻击，FP16 和 INT4 量化方案表现出相似的攻击特征，FP8 和 INT8 方案亦是如此。在层集中度方面（子图(a)和(b)），针对 FP16 和 INT4 模型的扰动更广泛地分布在各个层中。相反，对于 FP8 和 INT8 量化模型，成功的扰动倾向于集中在非常特定的、通常更狭窄的层范围内。关于目标模块（子图(c)和(d)），FP16 和 INT4 组中，多数扰动落在注意力机制组件内，特别是值投影（ v_proj ‘）模块。相比之下，FP8 和 INT8 组中，更大比例的修改针对 MLP 块组件，特别是在下投影（ down_proj ‘）模块中表现出显著集中。

6.6 攻击后量化

我们研究了在 FP16 模型中诱导的越狱状态在随后被量化的受感染模型中是否仍然存在。已越狱的 Llama-3.2-3B 和 Phi-4-mini 的 FP16 状态被量化为 FP8、INT8 和 INT4 权重格式，并在 HarmBench 上重新评估了语音识别（ASR）性能。表 4 中的结果表明，从 FP16 转移而来的越狱状态可以通过 8 位量化持续存在，而 4 位整数量化通过破坏这些现有的恶意扰动提供了更高的鲁棒性。复制原始 FP16 位翻转的精确数值影响

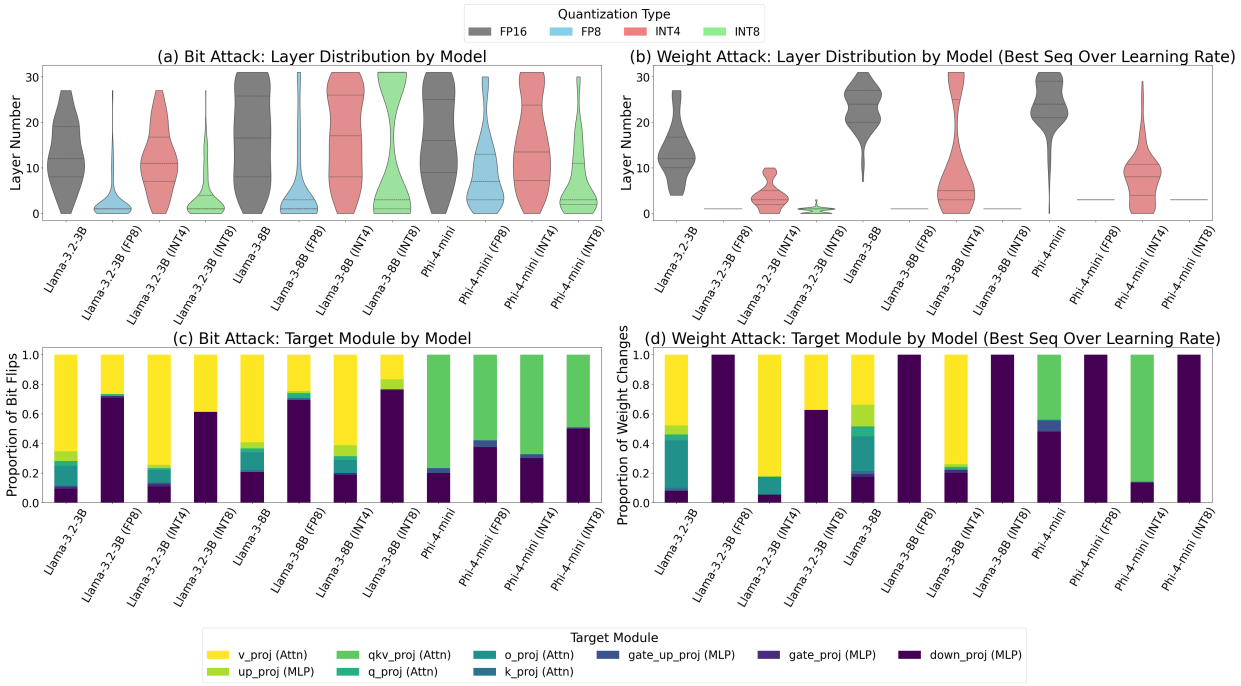


Figure 5: Distribution of perturbations locations across model layers and internal components.

directly onto an already quantized model would likely require a different, and potentially more extensive, set of bit manipulations due to the inherent differences in value representation and precision.

Table 4: ASR (%) (Δ vs. FP16) after Quantizing Jailbroken FP16 Models at 150 bit-flips.

Model	**FP16 Jailbroken**	**Post-Attack FP8**	**Post-Attack INT8**	**Post-Attack INT4**
Llama-3.2-3B	84.9	83.6 (-1.3)	84.3 (-0.6)	39.6 (-45.3)
Phi-4-mini	86.8	86.8 (0)	89.3 (+2.5)	62.3 (-24.3)

7 Conclusion

This paper investigated the vulnerability of aligned LMs to jailbreaking via targeted parameter manipulation, comparing bit-level and word-level attacks across FP16, FP8, INT8, and INT4 quantization on Llama-3.2-3B, Phi-4-mini, and Llama-3-8B. Our findings demonstrate that while FP16 models are readily jailbroken (>80% ASR within 25 perturbations), all tested quantization schemes significantly influence attack dynamics and ultimate success. Notably, the highly constrained Bit-Flip Attack proved to be as effective as the Word-Level Attack in achieving jailbreaks on FP16 models. FP8 quantization exhibited the strongest resilience, maintaining ASR below 65% even after 150 bit-flips, while INT8 also offered considerable protection. In contrast, INT4 quantization was significantly less resilient than the 8-bit formats. Analysis also revealed differing architectural attack targets based on quantization schemes. We observed significant variability in ASR between judges, which underlines the need for improved evaluation benchmarks. Furthermore, attack success demonstrated sensitivity to the specific samples chosen for optimization, showing potential for attack improvement. While direct attacks on quantized models proved challenging, jailbreaks induced in FP16 models were often transferable to subsequent 8-bit quantization, though INT4 substantially reduced this transferability. Future work should include physical fault injection and explore broader quantization and defense strategies.

References

[1] Gordon Burtch, Dokyun Lee, and Zhichen Chen. The consequences of generative ai for online knowledge communities. *Scientific Reports*, 14(1):10413, 2024.

复制原始 FP16 位翻转的精确数值影响，破解量化语言模型

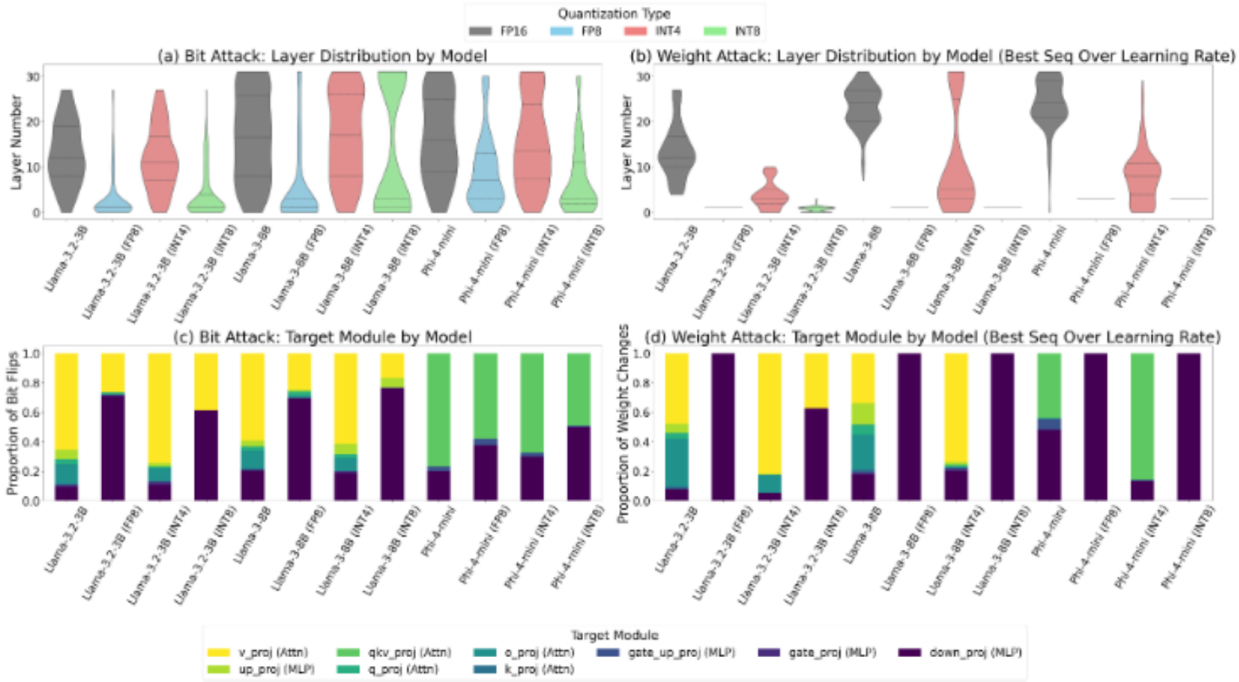


图 5：模型层和内部组件中扰动位置的分布。

直接应用于已量化的模型可能需要不同且可能更广泛的位操作，因为值表示和精度的固有差异。

表 4：在 150 位翻转后将越狱 FP16 模型量化后的 ASR（%） Δ vs. FP16）。

模型	FP16 后攻击	后攻击	后攻击	被越狱的 FP8	INT8	INT4
Llama-3.2-3B	84.9	83.6 (-1.3)	84.3 (-0.6)	39.6 (-45.3)		
Phi-4-mini	86.8	86.8 (0)	89.3 (+2.5)	62.3 (-24.3)		

7 结论

本文研究了对齐的大型语言模型（LLMs）在遭受针对性参数操控时的越狱漏洞，对比了在 Llama-3.2-3B、Phi-4-mini 和 Llama-3-8B 模型上 FP16、FP8、INT8 和 INT4 量化下的位级攻击和词级攻击。我们的研究发现，虽然 FP16 模型容易被越狱（在 25 次扰动内 ASR 超过 80%），所有测试的量化方案都显著影响了攻击动态和最终成功率。值得注意的是，高度受限的位翻转攻击在 FP16 模型上的越狱效果与词级攻击相当。FP8 量化表现出最强的抗性，即使经过 150 次位翻转，ASR 仍保持在 65%以下，而 INT8 也提供了相当的保护。相比之下，INT4 量化的抗性明显低于 8 位格式。分析还揭示了基于量化方案的不同架构攻击目标。我们观察到裁判间 ASR 存在显著差异，这突显了改进评估基准的必要性。此外，攻击成功率对优化所选择的特定样本敏感，显示出攻击改进的潜力。 在量化模型上的直接攻击被证明具有挑战性，FP16 模型中诱发的越狱攻击通常可以迁移到后续的 8 位量化，尽管 INT4 显著降低了这种可迁移性。未来的工作应包括物理故障注入，并探索更广泛的量化和防御策略。

参考文献

[1] Gordon Burtch, Dokyun Lee, and Zhichen Chen. 生成式人工智能对在线知识社区的影响。科学报告，14(1):10413，2024。

[2] Lexie Perrotta, Stefan Hennessy, and Grace Lee. The future of search: Ai-driven disruption and diversification. Industry report, AlixPartners LLP, 2025.

[3] Zhichao Wang, Bin Bi, Shiva Kumar Pentyala, Kiran Ramnath, Sougata Chaudhuri, Shubham Mehrotra, Zixu, Zhu, Xiang-Bo Mao, Sitaram Asur, Na, and Cheng. A comprehensive survey of llm alignment techniques: Rlhf, rlaif, ppo, dpo and more, 2024.

[4] Sibo Yi, Yule Liu, Zhen Sun, Tianshuo Cong, Xinlei He, Jiaxing Song, Ke Xu, and Qi Li. Jailbreak attacks and defenses against large language models: A survey, 2024.

[5] Jakub Łucki, Boyi Wei, Yangsibo Huang, Peter Henderson, Florian Tramèr, and Javier Rando. An adversarial perspective on machine unlearning for ai safety, 2025.

[6] Vladyslav Oles, Anna Schmedding, George Ostrouchov, Woong Shin, Evgenia Smirni, and Christian Engelmann. Understanding gpu memory corruption at extreme scale: The summit case study. In *Proceedings of the 38th ACM International Conference on Supercomputing*, pages 188–200, 2024.

[7] Francesco Regazzoni, Shivam Bhasin, Amir Ali Pour, Ihab Alshaer, Furkan Aydin, Aydin Aysu, Vincent Beroulle, Giorgio Di Natale, Paul Franzon, David Hely, et al. Machine learning and hardware security: Challenges and opportunities. In *Proceedings of the 39th International Conference on Computer-Aided Design*, pages 1–6, 2020.

[8] Jeremie S Kim, Minesh Patel, A Giray Yağlıkçı, Hasan Hassan, Roknoddin Azizi, Lois Orosa, and Onur Mutlu. Revisiting rowhammer: An experimental analysis of modern dram devices and mitigation techniques. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 638–651. IEEE, 2020.

[9] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. Flipping bits in memory without accessing them: An experimental study of dram disturbance errors. *ACM SIGARCH Computer Architecture News*, 42(3):361–372, 2014.

[10] Adnan Siraj Rakin, Zhezhi He, and Deliang Fan. Bit-flip attack: Crushing neural network with progressive bit search. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1211–1220, 2019.

[11] Adnan Siraj Rakin, Zhezhi He, Jingtao Li, Fan Yao, Chaitali Chakrabarti, and Deliang Fan. T-bfa: Targeted bit-flip adversarial weight attack. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(11):7928–7939, 2022.

[12] Huili Chen, Cheng Fu, Jishen Zhao, and Farinaz Koushanfar. Proflip: Targeted trojan attack with progressive bit flips. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7718–7727, 2021.

[13] Zachary Coalson, Jeonghyun Woo, Shiyang Chen, Yu Sun, Lishan Yang, Prashant Nair, Bo Fang, and Sanghyun Hong. Prisonbreak: Jailbreaking large language models with fewer than twenty-five targeted bit-flips. *arXiv preprint arXiv:2412.07192*, 2024.

[14] Jiedong Lang, Zhehao Guo, and Shuyu Huang. A comprehensive study on quantization techniques for large language models, 2024.

[15] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models, 2023.

[16] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation, 2013.

[17] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Huggingface's transformers: State-of-the-art natural language processing, 2020.

[18] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal, 2024.

[19] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[20] Prannaya Gupta, Le Qi Yau, Hao Han Low, I Lee, Hugo Maximus Lim, Yu Xin Teoh, Jia Hng Koh, Dar Win Liew, Rishabh Bhardwaj, Rajat Bhardwaj, et al. Walledeval: A comprehensive safety evaluation toolkit for large language models. *arXiv preprint arXiv:2408.03837*, 2024.

[2] Lexie Perrotta, Stefan Hennessy, 和 Grace Lee. 搜索的未来：AI 驱动的颠覆和多样化。
行业报告，AlixPartners LLP，2025 年。

[3] Zhichao Wang, Bin Bi, Shiva Kumar Pentyala, Kiran Ramnath, Sougata Chaudhuri, Shubham Mehrotra, Zixu Zhu, Xiang-Bo Mao, Sitaram Asur, Na, 和 Cheng. LLM 对齐技术的综合综述：RLHF, RLAIF, PPO, DPO 及其他，2024 年。[4] Sibo Yi, Yule Liu, Zhen Sun, Tianshuo Cong, Xinlei He, Jiaxing Song, Ke Xu, 和 Qi Li. 针对大型语言模型的越狱攻击与防御：一项综述，2024 年。[5] Jakub Łucki, Boyi Wei, Yangsibo Huang, Peter Henderson, Florian Tramèr, 和 Javier Rando. 从对抗视角看机器忘却对 AI 安全的影响，2025 年。

[6] Vladyslav Oles, Anna Schmedding, George Ostrouchov, Woong Shin, Evgenia Smirni, 和 Christian Engelmann.

理解极端规模下的 GPU 内存损坏：顶点案例研究。在 2024 年第 38 届 ACM 国际超级计算会议论文集中，第 188-200 页。

[7] Francesco Regazzoni, Shivam Bhasin, Amir Ali Pour, Ihab Alshaer, Furkan Aydin, Aydin Aysu, Vincent Beroulle, Giorgio Di Natale, Paul Franzon, David Hely, 等. 机器学习和硬件安全：挑战与机遇. 在第 39 届国际计算机辅助设计会议论文集中，第 1-6 页，2020 年. [8] Jeremie S Kim, Minesh Patel, A Giray Yağlıkçı, Hasan Hassan, Roknoddin Azizi, Lois Orosa, 和 Onur Mutlu.

重访 rowhammer：对现代 DRAM 设备及缓解技术的实验性分析。2020 年 ACM/IEEE 第 47 届年度国际计算机架构研讨会（ISCA），第 638-651 页。IEEE，2020。

[9] Kim Yoongu, Daly Ross, Kim Jeremie, Fallin Chris, Lee Ji Hye, Lee Donghyuk, Wilkerson Chris, Lai Konrad, 和 Mutlu Onur. 在不访问内存位的情况下翻转位：DRAM 扰动错误的实验研究. ACM SIGARCH 计算机架构新闻, 42(3):361–372, 2014.

[10] Adnan Siraj Rakin、Zhezhi He 和 Deliang Fan。比特翻转攻击：逐步用比特压垮神经网络搜索。在 IEEE/CVF 国际计算机视觉会议论文集中，第 1211-1220 页，2019 年。[11] Adnan Siraj Rakin，Zhezhi He，Jingtao Li，Fan Yao，Chaitali Chakrabarti 和 Deliang Fan。T-bfa：定向位翻转

对抗权重攻击。IEEE 图像分析及机器智能汇刊，44(11):7928–7939, 2022.

[12] 陈慧丽, 傅成, 赵吉森, 和库什万哈尔・法林纳兹. Proflip: 基于渐进位翻转的定向木马攻击. 在 IEEE/CVF 国际计算机视觉会议论文集，第 7718-7727 页，2021 年. [13] 扎卡里・科尔森, 吴正勋, 陈诗阳, 孙宇, 杨立山, 帕拉尚特・奈尔, 方波, 和宋 sanghyun

洪. Prisonbreak: 使用少于二十五次定向位翻转破解大型语言模型. arXiv 预印本 arXiv:2412.07192, 2024 年.

[14] 郎继东, 郭哲浩, 和黄舒宇. 大型语言模型量化技术的综合研究, 2024 年. [15] 安迪・邹, 王子帆, 尼古拉斯・卡林尼, 米拉德・纳斯尔, J. Zico・科尔特, 和马特・弗雷德里克森. 对齐语言模型的通用和可迁移对抗攻击, 2023 年. [16] 约书亚・本吉奥, 尼古拉斯・勒昂纳德, 和亚伦・库维. 通过随机神经元估计或传播梯度进行条件计算, 2013 年. [17] 托马斯・沃尔夫, 吕西安・德布, 维克多・桑, 朱利安・肖蒙, 克莱门特・德朗格, 安东尼・莫伊, 皮耶里克・西斯塔克,

Tim Rault，Rémi Louf，Morgan Funtowicz，Joe Davison，Sam Shleifer，Patrick von Platen，Clara Ma，Yacine Jernite，Julien Plu，Canwen Xu，Teven Le Scao，Sylvain Gugger，Mariama Drame，Quentin Lhoest，和 Alexander M. Rush. Huggingface 的 transformers：最先进的自然语言处理，2020。

[18] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart、Bo Li、David Forsyth 和 Dan Hendrycks。Harmbench：一个用于自动化红队测试和鲁棒拒绝的标准评估框架，2024 年。

[19] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, 等. Llama 3 模型群. arXiv 预印本 arXiv:2407.21783, 2024.

[20] Prannaya Gupta, Le Qi Yau, Hao Han Low, I Lee, Hugo Maximus Lim, Yu Xin Teoh, Jia Hng Koh, Dar Win Liew, Rishabh Bhardwaj, Rajat Bhardwaj, 等. Walledeval: 大型语言模型的全面安全评估工具包. arXiv 预印本 arXiv:2408.03837, 2024.